COMPUTER NETWORKS · COMPLETE REFERENCE

Transport Layer, in full detail

Every mechanism in Chapter 3, slide by slide · FSMs as transition tables · header fields bit by bit · the extras your professor is likely to ask about · a 30-problem practice bank · glossary

This is the companion to the shorter *Exam Prep* guide. That one tells you what to revise; this one has the full depth for when a question probes a detail. Slide references like `[3-72]` point to the Kurose & Ross Chapter 3 slides (v8.2). Purple **"Beyond the slides"** boxes cover the standard RFC and textbook material that the slides skip but examiners often ask about.

1	Transport-layer services, and how they relate to the network layer	2
2	Multiplexing and demultiplexing, ports and sockets	3
3	UDP and the Internet checksum	5
4	Principles of reliable data transfer: rdt1.0 → rdt3.0 with full FSM tables	7
5	Pipelined protocols: Go-Back-N and Selective Repeat in full	11
6	TCP: segment structure, sequence numbers, RTT, reliable transfer, fast retransmit	14
7	TCP flow control	18
8	TCP connection management: handshakes, closing, the state machine	18
9	Principles of congestion control	21
10	TCP congestion control: AIMD, slow start, the Reno FSM, CUBIC, BBR, ECN, fairness	23
11	Evolution of the transport layer: QUIC and friends	28
12	Sockets and tools: the Python socket API, blocking calls, Wireshark, ss, nmap	30
13	Transport-layer security issues	33
14	Practice problem bank (30 problems, with answers)	34
15	Rapid-fire recall: 60 one-line questions	40
16	Glossary	42

1 · Transport-layer services [3-2 … 3-9]

1.1 What the transport layer does

- It provides **logical communication between application processes** running on different hosts. “Logical” means that, from the application’s point of view, the hosts seem directly connected, even if they are on opposite sides of the planet with many routers and link types in between [3-4 notes].
- Transport protocols run **only in end systems**. Routers act only on network-layer (and lower) fields; they never look at transport-layer headers. (NATs and firewalls are the exception: these middleboxes *do* read ports.)
- **Sender actions** [3-7]: it is passed an application-layer message, determines the segment header field values, creates the segment, and passes the segment to IP.
- **Receiver actions** [3-8]: it receives the segment from IP, checks the header values, extracts the application-layer message, and demultiplexes it up to the application via a socket.
- The transport-layer PDU is a **segment**. In the UDP RFC it is sometimes called a *datagram*, but K&R use “segment” for both TCP and UDP and reserve “datagram” for the network layer.

1.2 Transport layer vs network layer [3-5, 3-6]

	Network layer (IP)	Transport layer (TCP/UDP)
Communication between	**Hosts** (identified by IP address)	**Processes** (identified by IP address + port)
Runs in	Hosts *and* routers	Hosts only
Service	Best effort: no guarantee of delivery, order or integrity	UDP adds process-to-process delivery and error checking. TCP adds reliability, ordering, flow control and congestion control
Constraint	—	Can only *enhance* the network service. It cannot add delay or bandwidth guarantees the network does not provide, although it *can* provide reliability and encryption over an unreliable network

Household analogy (K&R textbook): 12 kids in Ann’s house send letters to 12 kids in Bill’s house. The **houses are hosts**, the **kids are processes**, the **letters are application messages**, **Ann and Bill are the transport protocol** (they collect and distribute letters within each house), and the **postal service is the network-layer protocol**. If Ann and Bill fall ill and are replaced by Susan and Harvey (who may lose letters), that is a different transport protocol over the *same* network service. Ann and Bill cannot promise a delivery time that the post office does not provide.

1.3 The two Internet transport protocols [3-9]

Service	TCP	UDP
Process-to-process delivery (mux/demux)	Yes	Yes
Error detection (checksum)	Yes (mandatory)	Yes (optional in IPv4, mandatory in IPv6)
Reliable, in-order delivery	Yes: ACKs, sequence numbers, timers, retransmission	No: segments may be lost or reordered
Flow control	Yes (rwnd)	No
Congestion control	Yes (cwnd)	No
Connection setup	Yes: 3-way handshake, with state at both ends	No: connectionless, and each segment is independent

Data model	Byte stream with no message boundaries	Message-oriented: one send = one datagram, and boundaries are preserved
Header	20–60 bytes	8 bytes
Communication	Point-to-point (unicast only)	Unicast, **multicast and broadcast** possible
Not provided by either	**Delay guarantees, bandwidth (throughput) guarantees**, and security (TLS adds security on top of TCP; QUIC has it built in)	

BEYOND THE SLIDES · Four dimensions of transport service (from Chapter 2)

Applications need some combination of **reliable data transfer**, **throughput**, **timing** and **security**. The Internet offers only reliability (TCP) and security (TLS/QUIC). Timing and throughput are left to the application: DASH adapts its bitrate, VoIP uses playout buffers, and CDNs reduce RTT. This is a standard 2-mark question: “Why can’t TCP guarantee a minimum throughput?” Because IP is best effort and shares links with competing traffic, and a transport protocol cannot create capacity that the network does not provide.

2 · Multiplexing and demultiplexing [3-11 ... 3-27]

- **Multiplexing at the sender**: gather data chunks from many sockets and encapsulate each with a transport header that carries the source and destination port numbers (used later for demultiplexing).
- **Demultiplexing at the receiver**: use the header information to deliver each received segment to the correct socket. The slides frame this as: “how did the transport layer know to deliver a message to the Firefox process rather than Netflix or Skype?” [3-13]
- **How it works** [3-22]: the host receives IP datagrams, each carrying source and destination IP addresses and **one** transport segment. Each segment carries source and destination **port numbers** (16 bits each). The host uses **IP addresses and port numbers** to direct the segment to the right socket.
- Multiplexing and demultiplexing happen at **every layer** [3-27]: the Ethernet type field chooses IPv4/IPv6/ARP, the IP protocol field chooses TCP (6), UDP (17) or ICMP (1), and the ports choose the socket.

2.1 Port numbers

Range	Name	Examples
0 – 1023	**Well-known** (reserved, RFC 1700 / IANA; binding needs root on Unix)	20/21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 110 POP3, 143 IMAP, 161/162 SNMP, 443 HTTPS & QUIC
1024 – 49151	Registered	3306 MySQL, 5060 SIP, 8080 HTTP-alt
49152 – 65535	Dynamic / **ephemeral**	Chosen automatically by the OS for the client side of a connection

A **server** binds a well-known port so that clients can find it. A **client** usually lets the OS pick an ephemeral port. If the client does not call bind(), the port is assigned automatically at connect() or at the first sendto().

2.2 Connectionless (UDP) demultiplexing [3-23, 3-24]

- When creating a socket you must specify a **host-local port #**, e.g. `DatagramSocket mySocket1 = new DatagramSocket(12534);`
- When sending, you must specify the destination IP address and the destination port #.

- The receiving host checks the **destination port # only** and directs the segment to the socket bound to that port.
- **Key point**: IP/UDP datagrams with the *same destination port* but *different source IP addresses and/or source ports* go to the **same socket**. The source port serves as a "return address", and the application reads it with recvfrom().
- Slide example [3-24]: the server's UDP socket is bound to port 6428, and clients use sockets on ports 9157 and 5775. A segment from client A has *source port 9157, dest port 6428*; the server's reply has *source port 6428, dest port 9157*. Replies swap the ports.

2.3 Connection-oriented (TCP) demultiplexing [3-25, 3-26]

- A TCP socket is identified by a **4-tuple**: source IP, source port, destination IP, destination port. The receiver uses **all four** values to demultiplex.
- A server supports many simultaneous TCP sockets, each with its own 4-tuple and each associated with a different client. For example, a web server has one connection socket per client connection. **Non-persistent HTTP** creates and closes a new socket for *every* request, which is expensive.
- Slide example [3-26]: three segments are all destined to IP B, port 80, but they are demultiplexed to **different** sockets (P4, P5, P6) because their source IP or source port differs. Two clients on different hosts may even use the *same* source port, since the source IP differs.
- The **welcoming (listening) socket** is identified only by the local (IP, port). It receives SYNs, and accept() creates a new connection socket with the full 4-tuple. A modern server may use one process per connection or, more commonly, one thread or event loop per connection.

> **BEYOND THE SLIDES · Port scanning (security link)**
>
> Because demultiplexing is by port, an attacker can probe which ports are open. **nmap** sends a TCP SYN to each port: a **SYNACK** means open, **RST** means closed, and no reply means filtered. For UDP, silence may mean open, and an **ICMP port unreachable** reply means closed. Knowing that a port is open reveals which application is running and which exploits might apply.

3 · UDP: User Datagram Protocol [3-29 … 3-40] RFC 768

3.1 Service

- A "**no frills**, bare bones" Internet transport protocol offering a **best-effort** service: segments may be **lost** or **delivered out of order** to the application.
- **Connectionless**: there is no handshaking between UDP sender and receiver, and each segment is handled independently of the others.
- RFC 768 is only about 2.5 pages long [3-31 notes].

3.2 Why UDP exists [3-30, 3-40]

1. **No connection establishment**, so no RTT of delay is incurred. This is why DNS uses UDP, and a large part of why QUIC uses UDP.
2. **Simple**: there is no connection state at the sender or receiver (no buffers, sequence numbers or congestion parameters), so a server can support many more active clients.
3. **Small header**: 8 bytes vs 20 bytes for TCP.
4. **No congestion control**: UDP can blast away as fast as desired, and **can function when the network service is compromised**. This is also why it is a fairness concern.
5. **Finer application-level control** over what data is sent and when: data goes immediately into a segment, with no Nagle-style buffering or retransmission delay.

3.3 Where UDP is used

Application	Application protocol	Transport	Why
Name translation	DNS	UDP (TCP for large replies and zone transfers)	1 RTT, stateless server
Network management	SNMP	UDP	Must work when the network is stressed
Routing	RIP	UDP	Periodic, loss-tolerant updates
Streaming multimedia, Internet telephony	RTP, proprietary	UDP (or TCP for DASH)	Loss-tolerant, rate-sensitive
Web (HTTP/3)	QUIC	UDP	Reliability and congestion control implemented at the application layer
Address configuration	DHCP	UDP	The client has no IP address yet, so it must broadcast
Email, remote login, web (HTTP/1.1, /2), file transfer	SMTP, SSH, HTTP, FTP	TCP	Need reliability

If reliable transfer is needed over UDP (as in HTTP/3), the application must add **reliability** and **congestion control** at the application layer [3-30].

3.4 UDP segment header [3-35]

Bits 0–15	Bits 16–31
Source port #	Dest port #
Length (bytes of UDP segment, **including header**)	Checksum

Application data (payload)

- The length field is needed because the payload size varies from segment to segment. Its minimum value is 8 (header only). The maximum payload over IPv4 is 65,535 − 8 − 20 = **65,507 bytes**.
- The source port is optional (0 if not used), but it is needed if the receiver is to reply.

3.5 The Internet checksum [3-36 … 3-39]

- **Goal**: detect errors (flipped bits) in the transmitted segment.
- **Intuition** [3-36]: if the sender transmits 5, 6 and their sum 11, and the receiver gets 4, 6, 11, then 4 + 6 ≠ 11, so an error is detected.
- **Sender**: treat the segment contents, *including the UDP header fields and the IP addresses*, as a sequence of 16-bit integers. The checksum is the **1's complement of the 1's-complement sum** of those integers, and it is placed in the checksum field.
- **Receiver**: compute the checksum of the received segment. If it is **not equal** to the field, an error is detected. If it is equal, no error is detected, *"but maybe errors nonetheless?"*. Equivalently, add everything including the checksum: the result must be 1111…1.
- **Wraparound**: any carry out of the most significant bit is added back into the result.

```
  1110 0110 0110 0110
+ 1101 0101 0101 0101
──────────────────────
1 1011 1011 1011 1011      ← 17th bit is a carry
  1011 1011 1011 1100      ← wraparound: add the carry to the LSB   (sum)
  0100 0100 0100 0011      ← invert every bit                        (checksum)
```

- **Weak protection** [3-39]: flip one bit in each word in the same position (e.g. the last bit of word 1 goes 0→1 and the last bit of word 2 goes 1→0) and the sum, and therefore the checksum, is **unchanged**. Swapping the order of 16-bit words is also undetectable.

BEYOND THE SLIDES · Checksum details examiners like

- **Pseudo-header**: the UDP and TCP checksums also cover a 12-byte pseudo-header containing the source IP, destination IP, a zero byte, the protocol number (17 = UDP, 6 = TCP) and the UDP/TCP length. This detects segments delivered to the *wrong host*. It technically violates layering, which is one reason NAT must recompute transport checksums.
- UDP checksum = 0 means "no checksum" in IPv4. In IPv6 the checksum is **mandatory**, because the IPv6 header has no checksum of its own.
- **Why check at the transport layer if links have CRCs?** Not every link on the path may check errors, and bits can be corrupted inside router memory. This is the **end-to-end principle**: some functions must be done end to end, and lower-layer checks are only a performance optimisation.
- UDP does error *detection*, not recovery. A corrupted segment is discarded, or passed up with a warning in some implementations.
- Why 1's complement? It is byte-order independent, cheap to compute incrementally (routers can update it after changing a field), and the carries wrap around so no information is lost.

4 · Principles of reliable data transfer [3-42 … 3-69]

4.1 The problem and the interfaces

- The **service abstraction** is a reliable channel: data sent by the sending process arrives, uncorrupted and in order, at the receiving process. The **implementation** is a sender-side and a receiver-side rdt protocol running over an **unreliable channel**. How complex the protocol must be depends on the channel's **impairments** (corruption, loss, reordering) [3-42 … 3-44].
- Although the data flows one way, **control information flows both ways** [3-43 notes]: the sender and receiver exchange messages to implement one-way reliable transfer.
- **The "curtain"** [3-45 notes]: neither side knows the other's state. The sender learns whether its message arrived *only* if the receiver tells it. This is the whole reason a protocol is needed.

Interface call	Called by	Meaning
`rdt_send(data)`	Upper layer (application) at the sender	Pass data to be delivered reliably
`udt_send(packet)`	rdt at the sender (and at the receiver, for ACKs)	Send a packet over the **u**nreliable channel
`rdt_rcv(packet)`	Called when a packet arrives on the receive side of the channel	A packet has arrived
`deliver_data(data)`	rdt at the receiver	Deliver data up to the application

FSM notation [3-47]: a transition is labelled *event* over a line, with the *actions* below it. Λ means "no event" or "no action". In a given state, the next state is uniquely determined by the next event. Unidirectional data transfer is assumed throughout.

4.2 rdt1.0: reliable transfer over a reliable channel [3-48]

The underlying channel is perfectly reliable: no bit errors and no loss. The sender and receiver each have **one state**. Sender: `rdt_send(data) / packet = make_pkt(data); udt_send(packet)`. Receiver: `rdt_rcv(packet) / extract(packet,data); deliver_data(data)`. No feedback is needed.

4.3 rdt2.0: channel with bit errors [3-49 … 3-55]

The channel may flip bits, and a **checksum** detects them. Recovery uses ARQ (Automatic Repeat reQuest): **ACKs** ("received OK"), **NAKs** ("had errors") and **retransmission** on a NAK. It is the human analogy of saying "OK" or "please repeat that".

State	Event	Action	Next state
Sender (2 states)			
Wait for call from above	`rdt_send(data)`	`sndpkt = make_pkt(data, checksum); udt_send(sndpkt)`	Wait for ACK or NAK
Wait for ACK or NAK	`rdt_rcv(rcvpkt) && isNAK(rcvpkt)`	`udt_send(sndpkt)`	Wait for ACK or NAK
Wait for ACK or NAK	`rdt_rcv(rcvpkt) && isACK(rcvpkt)`	Λ	Wait for call from above
Receiver (1 state)			
Wait for call from below	`rdt_rcv(rcvpkt) && corrupt(rcvpkt)`	`udt_send(NAK)`	Same
Wait for call from below	`rdt_rcv(rcvpkt) && notcorrupt(rcvpkt)`	`extract(rcvpkt,data); deliver_data(data); udt_send(ACK)`	Same

- While waiting for an ACK or NAK, the sender **cannot accept new data** from above. This makes rdt2.0 a **stop-and-wait** protocol.
- The receiver's state ("did I get it correctly?") is unknown to the sender unless it is communicated. *"That's why we need a protocol!"* [3-52]
- **Fatal flaw** [3-55]: the ACK or NAK itself may be corrupted. The sender then does not know what happened at the receiver, and it cannot simply retransmit because that may create a **duplicate**. The alternatives, such as NAK-ing the NAK or adding enough checksum bits to *correct* errors, lead to an infinite regress or are expensive. The fix is to **retransmit on a garbled ACK/NAK and add sequence numbers** so the receiver can discard duplicates.

4.4 rdt2.1: handling garbled ACK/NAKs [3-56 ... 3-58]

State	Event	Action	Next state
Sender (4 states; the transitions for seq 1 mirror seq 0)			
Wait for call 0 from above	`rdt_send(data)`	`sndpkt = make_pkt(0, data, checksum); udt_send(sndpkt)`	Wait for ACK or NAK 0
Wait for ACK or NAK 0	`rdt_rcv && (corrupt(rcvpkt) || isNAK(rcvpkt))`	`udt_send(sndpkt)`	Same
Wait for ACK or NAK 0	`rdt_rcv && notcorrupt && isACK`	Λ	Wait for call 1 from above
Receiver (2 states: "wait for 0 from below" and "wait for 1 from below")			
Wait for 0 from below	`rdt_rcv && notcorrupt && has_seq0`	`extract; deliver_data; sndpkt = make_pkt(ACK, chksum); udt_send`	Wait for 1 from below
Wait for 0 from below	`rdt_rcv && corrupt(rcvpkt)`	`sndpkt = make_pkt(NAK, chksum); udt_send`	Same
Wait for 0 from below	`rdt_rcv && notcorrupt &&` **`has_seq1`**	`sndpkt = make_pkt(ACK, chksum); udt_send` **`(ACK the duplicate, do not deliver it)`**	Same

- **Sender**: adds a seq # to each packet; **two seq #s (0, 1) suffice**; must check whether a received ACK/NAK is corrupted; has **twice as many states**, because the state must "remember" whether the expected packet has seq # 0 or 1.
- **Receiver**: must check whether a received packet is a duplicate, and its state indicates whether 0 or 1 is the expected seq #. Note that the receiver **cannot know** whether its last ACK/NAK was received OK at the sender.
- **Why ACK a duplicate?** A duplicate means the sender never received our ACK. If we stayed silent, the sender would retransmit forever, which is a deadlock.

4.5 rdt2.2: a NAK-free protocol [3-59, 3-60]

- It has the same functionality as rdt2.1 but uses **ACKs only**. Instead of a NAK, the receiver sends an ACK for the **last packet received OK**, and it must **explicitly include the seq #** of the packet being ACKed.
- A **duplicate ACK** at the sender causes the same action as a NAK: **retransmit the current packet**. *TCP uses this approach to be NAK-free.*

Side / state	Event	Action	Next
Sender: wait for ACK 0	`rdt_rcv && (corrupt || isACK(rcvpkt, 1))`	`udt_send(sndpkt)`	Same
Sender: wait for ACK 0	`rdt_rcv && notcorrupt && isACK(rcvpkt, 0)`	Λ	Wait for call 1

Receiver: wait for 0 from below	`rdt_rcv && notcorrupt && has_seq0`	`extract; deliver; sndpkt = make_pkt(ACK, 0, chksum); udt_send`	Wait for 1
Receiver: wait for 0 from below	`rdt_rcv && (corrupt || has_seq1)`	`udt_send(sndpkt)` (re-sends **ACK 1**, i.e. the last good packet)	Same

4.6 rdt3.0: channels with errors *and loss* [3-61 … 3-66]

- New channel assumption: the channel can also **lose** packets, both data and ACKs. Checksums, seq #s, ACKs and retransmissions help, but they are *not quite enough*: if a packet is lost, the sender would wait forever.
- Approach: the sender waits a "**reasonable**" amount of time for an ACK and retransmits if none arrives. A **countdown timer** interrupts it after that time. Ideally the wait is at least RTT plus the processing time at the receiver.
- If the packet or ACK was only **delayed** (not lost), the retransmission is a duplicate, but seq #s already handle this. The receiver must specify the seq # of the packet being ACKed.
- **The rdt3.0 receiver is identical to the rdt2.2 receiver.** Only the sender changes.
- rdt3.0 is also called the **alternating-bit protocol**, because packet seq #s alternate 0, 1, 0, 1, ...

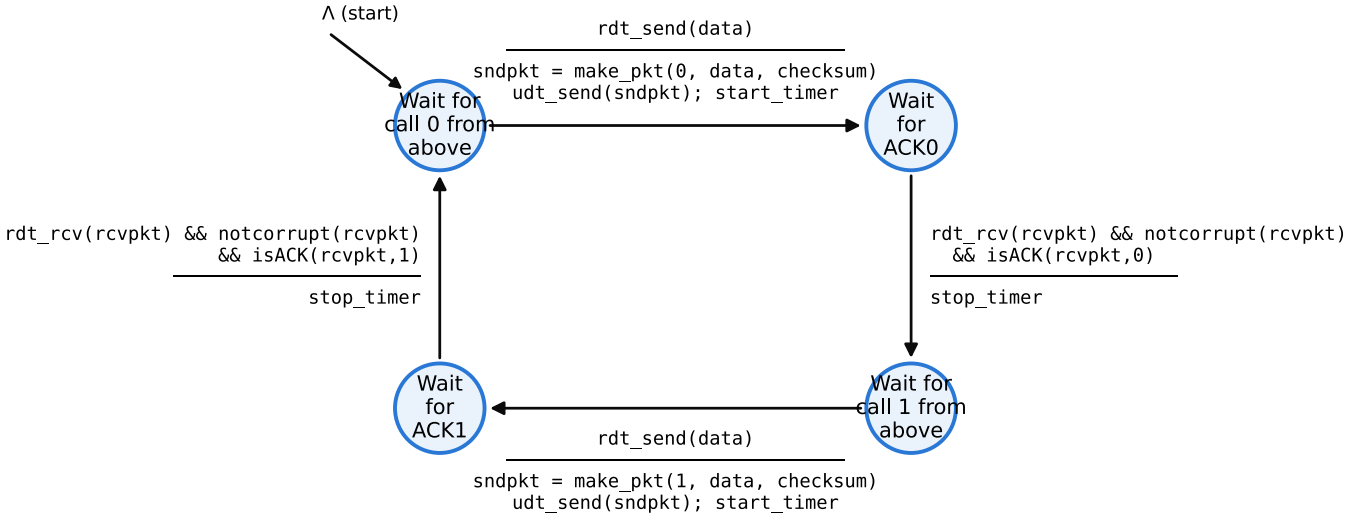


Figure 1. rdt3.0 sender: four states, the same as rdt2.1/2.2 [3-63 notes], plus a timer. Self-loops not drawn: in the *Wait for ACK* states, `timeout → udt_send(sndpkt); start_timer` and `corrupt(rcvpkt) || wrong ACK # → Λ`. In the *Wait for call* states, `rdt_rcv(rcvpkt) → Λ` (late ACKs are ignored).

> **Subtle difference from rdt2.2 (a favourite exam trap)**
> In rdt2.2 the sender **retransmits immediately** on a corrupt or duplicate ACK. In rdt3.0 the sender **does nothing** (Λ) on a corrupt or wrong ACK and just waits for the timeout. Reacting to both a duplicate ACK and the timeout would send extra copies, which could multiply after a premature timeout (the "sorcerer's apprentice" problem).

The four rdt3.0 scenarios [3-65, 3-66]

Scenario	What happens
(a) No loss	pkt0 → ack0 → pkt1 → ack1 → pkt0 … one packet per RTT.
(b) Packet loss	pkt1 is lost, so there is no ACK. The **timeout** fires and pkt1 is resent. The receiver gets pkt1 and sends ack1.
(c) ACK loss	pkt1 arrives but ack1 is lost. The timeout fires and pkt1 is resent. The receiver **detects a duplicate** (it expects 0), discards it and **re-sends ack1**.

(d) Premature timeout / delayed ACK	ack1 is merely slow. The timer fires and pkt1 is resent. The original ack1 then arrives, so the sender moves on and sends pkt0. The receiver detects the duplicate pkt1 and re-sends ack1. The sender gets a second ack1 while waiting for ack0 and **ignores it**. The protocol still works, just wastefully.

4.7 Performance of rdt3.0 (stop-and-wait) [3-67 ... 3-69]

```
U_sender = (L/R) / (RTT + L/R)
Example: R = 1 Gbps, one-way propagation = 15 ms (RTT = 30 ms), L = 8000 bits
  D_trans = L/R = 8000 / 10^9 = 8 µs
  U = 0.008 / 30.008 = 0.00027
  Throughput = 8000 bits / 30.008 ms ≈ 267 kbps   (a 1 KB packet every 30 ms ≈ 33 kB/s)
```

“rdt3.0 protocol performance stinks! The protocol limits the performance of the underlying infrastructure (channel).” The first bit reaches the receiver at t = RTT/2 (ignoring L/R), the last bit at RTT/2 + L/R, and the ACK is back at RTT + L/R. The sender is idle almost the whole time.

4.8 Summary: reliable data transfer mechanisms

Mechanism	Use / comments
Checksum	Detect bit errors in a packet
Timer	Time out and retransmit a packet whose packet or ACK was lost. A premature timeout (delay rather than loss) creates duplicates
Sequence number	Number the packets in order. A gap lets the receiver detect loss, and a repeated number lets it detect duplicates
Acknowledgement	Tell the sender that a packet (or set of packets) arrived correctly. It may be individual or cumulative, and it carries the seq # of what it acknowledges
Negative ACK	Tell the sender a packet was *not* received correctly. It carries the seq # of that packet. It is optional, since a duplicate ACK can serve instead
Window, pipelining	Allow up to N unACKed packets in flight. N may be set by the receiver’s buffer space (flow control) or by the level of congestion (congestion control)

5 · Pipelined protocols: Go-Back-N and Selective Repeat [3-70 ... 3-80, 3-148, 3-149]

Pipelining [3-70] means the sender allows multiple "in-flight", not-yet-acknowledged packets. It requires (1) a larger range of seq #s and (2) buffering at the sender and/or receiver. With 3 packets in flight, utilisation triples: U = 3L/R / (RTT + L/R) = 0.0008 [3-71]. The two generic forms are **Go-Back-N** and **Selective Repeat** [3-71 notes].

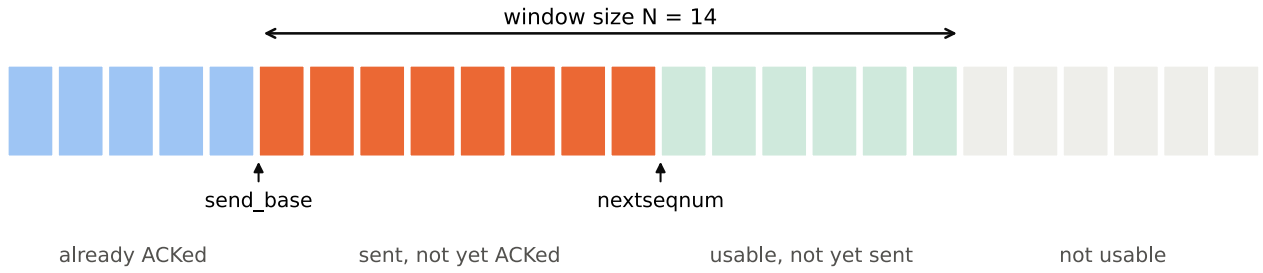


Figure 2. GBN sender's view of sequence numbers. In the slide example [3-72 notes], the window size is 14, 8 packets have been sent but not ACKed, and 6 sequence numbers are usable but not yet used, since no data has come from above.

5.1 Go-Back-N sender [3-72, 3-148]

- A "window" of up to **N consecutive transmitted but unACKed packets**, with a **k-bit seq #** in the packet header. Seq #s are computed modulo 2^k.
- **Cumulative ACK**: ACK(n) acknowledges all packets up to *and including* seq # n. On receiving ACK(n), the window moves forward to begin at n + 1.
- There is a **timer for the oldest in-flight packet**. **timeout(n)**: retransmit packet n *and all higher seq # packets* in the window. This is where "go back N" comes from.
- This is also called a **sliding-window protocol**.

Event	Action (extended FSM; initially base = 1, nextseqnum = 1)
rdt_send(data)	`if (nextseqnum < base + N) { sndpkt[nextseqnum] = make_pkt(nextseqnum, data, chksum); udt_send(sndpkt[nextseqnum]); if (base == nextseqnum) start_timer; nextseqnum++ } else refuse_data(data)`
timeout	`start_timer; udt_send(sndpkt[base]); udt_send(sndpkt[base+1]); … udt_send(sndpkt[nextseqnum−1])`
rdt_rcv && notcorrupt	`base = getacknum(rcvpkt) + 1; if (base == nextseqnum) stop_timer else start_timer`
rdt_rcv && corrupt	Λ

refuse_data: in a real implementation the sender buffers the data or signals the upper layer (a flag or semaphore) that the window is full.

5.2 Go-Back-N receiver [3-73, 3-149]

- **ACK-only**: always send an ACK for the correctly received packet with the **highest in-order** seq # so far. This may generate duplicate ACKs, and the receiver need only remember `expectedseqnum` (rcv_base).
- On receipt of an out-of-order packet it **can discard it or buffer it**, which is an implementation decision. Either way it re-ACKs the packet with the highest in-order seq #. The textbook FSM discards, so no receiver buffering is needed.

Event	Action (initially expectedseqnum = 1, sndpkt = make_pkt(0, ACK, chksum))
rdt_rcv && notcorrupt && hasseqnum(rcvpkt, expectedseqnum)	`extract(rcvpkt, data); deliver_data(data); sndpkt = make_pkt(expectedseqnum, ACK, chksum); udt_send(sndpkt); expectedseqnum++`
default (corrupt or out of order)	`udt_send(sndpkt)  (re-ACK the last in-order packet)`

GBN in action [3-74] (N = 4, pkt2 lost)

The sender sends 0, 1, 2, 3 and waits. The receiver gets 0 (ACK0) and 1 (ACK1). pkt2 is lost. The receiver gets 3, **discards** it and re-sends ACK1. ACK0 and ACK1 slide the window, so the sender sends 4 and 5, which the receiver discards, re-sending ACK1 each time. The sender **ignores the duplicate ACKs**. The pkt2 timeout fires, and the sender resends **2, 3, 4, 5**. The receiver now delivers 2, 3, 4, 5, sending ACK2 … ACK5.

5.3 Selective Repeat [3-75 ... 3-78]

- The receiver **individually ACKs** all correctly received packets and **buffers** packets as needed for eventual in-order delivery to the upper layer.
- The sender maintains (conceptually) a **timer for each unACKed packet**, and on a timeout retransmits **only that packet**. It keeps a window over N consecutive seq #s, which limits the pipelined packets to that window.
- The **sender and receiver windows are not always aligned**. Their views of which packets have been received differ.

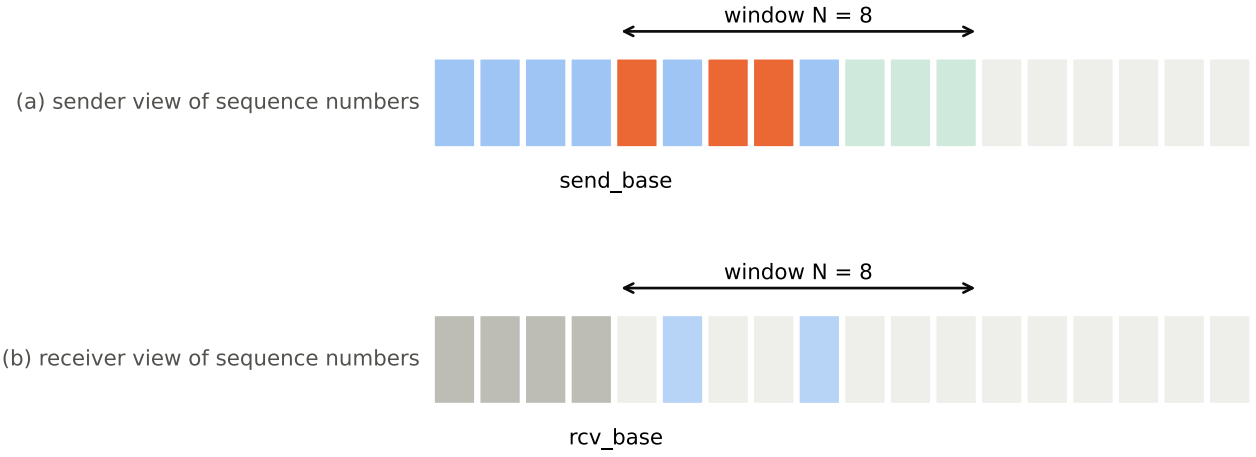


Figure 3. Selective Repeat windows. Sender: blue = ACKed, orange = sent but not ACKed, green = usable but not yet sent. Receiver: dark grey = already delivered, light blue = received out of order (buffered and ACKed), pale = acceptable (within the window).

SR sender	
Data from above	If the next available seq # is in the window, send the packet
timeout(n)	Resend packet n and restart its timer
ACK(n) in [sendbase, sendbase+N−1]	Mark packet n as received. If n is the smallest unACKed packet, advance the window base to the next unACKed seq #, and transmit any unsent packets that are now inside the window

SR receiver	
pkt n in [rcvbase, rcvbase+N−1]	Send ACK(n). If out of order, buffer it. If in order, deliver it (together with any buffered in-order packets) and advance the window to the next not-yet-received packet
pkt n in [rcvbase−N, rcvbase−1]	**Send ACK(n)** even though it was already ACKed. Otherwise the sender, whose ACK was lost, never advances its window
Otherwise	Ignore

SR in action [3-78] (N = 4, pkt2 lost)

The receiver ACKs 0 and 1, then gets 3, buffers it and sends ACK3. The sender records that ACK3 has arrived and, when the window allows, sends 4 and 5. The receiver buffers them and sends ACK4 and ACK5. The pkt2 timeout fires and the sender resends **only 2**. The receiver gets 2 and delivers 2, 3, 4 and 5 together, sending ACK2. **"What happens when ack2 arrives?"** The sender marks 2 as ACKed. Since 3, 4 and 5 are already ACKed, send_base jumps to 6 and the sender can transmit 6, 7, 8 and 9.

5.4 The Selective Repeat dilemma: window vs seq # space [3-79, 3-80]

Take seq #s 0, 1, 2, 3 (4 values) and a window of size 3.

(a) No problem	(b) Oops!
pkt0, 1 and 2 are sent, received and ACKed, and the receiver's window moves to {3, 0, 1}. All ACKs arrive. The sender sends pkt3 (lost) and then a **new pkt0**. The receiver accepts pkt0, which is correct.	pkt0, 1 and 2 are received and the receiver's window moves to {3, 0, 1}, but **all ACKs are lost**. The sender times out and **retransmits the old pkt0**. The receiver's window contains 0, so it accepts old data as new. **Wrong!**

The receiver cannot see the sender's side, and its behaviour is identical in both cases.

Something is very wrong. Requirement: **window size ≤ ½ × (size of the seq # space)**, i.e. $N \leq 2^{k-1}$. For GBN the requirement is $N \leq 2^k - 1$, because the GBN receiver window has size 1.

> **BEYOND THE SLIDES · Why these bounds hold (short proofs)**
>
> - **SR**: at any moment, the sender's window base is somewhere in [rcv_base − N, rcv_base], so the numbers the receiver might see are [rcv_base − N, rcv_base + N − 1], which is **2N** distinct values. These must not wrap, so $2N \leq 2^k$.
> - **GBN**: the receiver accepts only expectedseqnum. The sender's possible window spans [expected − N, expected + N − 1], and the old packet expected − N must not share a number with expected, so $N + 1 \leq 2^k$.
> - **Reordering in the network**: packets can also be *delayed* and reappear later. Real protocols therefore assume a **maximum packet lifetime** (TCP assumes about 3 minutes, the MSL) and make sure seq #s are not reused within it.

5.5 GBN vs SR at a glance

	Stop-and-wait (rdt3.0)	Go-Back-N	Selective Repeat	TCP
Sender window	1	N	N	min(cwnd, rwnd) bytes
Receiver window	1	1	N	rwnd (buffers out-of-order data)
ACKs	Per packet	Cumulative	Individual	Cumulative (+ SACK option)
Timers	1	1 (oldest)	1 per packet	1 (oldest unACKed)
Retransmits	The packet	Whole window from n	Only the lost one	Only the oldest unACKed segment
Seq # bound	2 values	$N \leq 2^k - 1$	$N \leq 2^{k-1}$	32-bit byte numbers

6 · TCP: connection-oriented transport [3-82 … 3-93] RFCs 793 (now 9293), 1122, 2018, 5681, 6298, 7323

6.1 Overview [3-82]

Point-to-point	One sender and one receiver. No multicast.
Reliable, in-order byte stream	No “message boundaries”. Two 100-byte writes may be read as one 200-byte read, so an application must frame its own messages (e.g. with HTTP’s Content-Length).
Full duplex	Bi-directional data flow in the same connection.
MSS	Maximum segment size: the maximum *application data* per segment, excluding headers. Typically **1460 bytes** (1500-byte Ethernet MTU − 20 IP − 20 TCP) [3-82 notes]. Announced in the SYN’s MSS option.
Cumulative ACKs	An ACK acknowledges everything up to the byte before the ACK number.
Pipelining	TCP congestion and flow control set the window size.
Connection-oriented	Handshaking (the exchange of control messages) initialises sender and receiver state before data is exchanged. Only the end systems keep connection state; routers are unaware of TCP connections.
Flow controlled	The sender will not overwhelm the receiver.

The application writes data into the **TCP send buffer** (set aside during the handshake). TCP takes chunks of at most MSS from it, adds a header, and passes each segment to IP. The receiver places data in the **receive buffer**, from which the application reads.

6.2 TCP segment structure [3-83]

Field	Size	Meaning
Source port, dest port	16 + 16 b	Demultiplexing (with the IP addresses: the 4-tuple)
Sequence number	32 b	Counts **bytes of data**, not segments: the byte-stream number of the first data byte in this segment
Acknowledgement number	32 b	Seq # of the **next byte expected** from the other side. Valid only if A = 1
Header length	4 b	In 32-bit words: minimum 5 (20 B), maximum 15 (**60 B**), so options take at most 40 B
Not used / reserved	4 b	Must be zero
C, **E**	1 b each	CWR (Congestion Window Reduced) and ECE (ECN-Echo): congestion notification
U (URG)	1 b	Urgent data present; the urgent pointer is valid (rarely used)
A (ACK)	1 b	The ACK number is valid (set on every segment after the SYN)
P (PSH)	1 b	Push: pass data to the application now (rarely used in practice)
R (RST), **S** (SYN), **F** (FIN)	1 b each	Connection management: abort, establish, close
Receive window	16 b	**Flow control**: the number of bytes the receiver is willing to accept (rwnd)
Checksum	16 b	Internet checksum over the header, data and pseudo-header (as in UDP)

Urgent data pointer	16 b	Offset of the last byte of urgent data
Options	0–40 B	See below

BEYOND THE SLIDES · TCP options you should know

MSS (SYN only)	Each side announces the largest segment it can receive
Window scale (RFC 7323, SYN only)	A shift count (0–14) applied to rwnd, giving a window of up to 65,535 × 2^{14} ≈ 1 GB. Needed when the bandwidth-delay product exceeds 64 KB
SACK-permitted / SACK (RFC 2018)	The receiver reports up to 3–4 blocks of out-of-order bytes it holds, so the sender retransmits only the holes. This makes TCP behave much like Selective Repeat
Timestamps (RFC 7323)	Accurate RTT measurement, even for retransmitted segments, and **PAWS** (Protection Against Wrapped Sequence numbers) on fast links
TCP Fast Open (RFC 7413)	A cookie lets a returning client carry data in the SYN, saving 1 RTT

6.3 Sequence numbers and ACKs [3-84, 3-85]

- The sender's sequence-number space is divided into: **sent and ACKed** · **sent, not yet ACKed ("in flight")** · **usable but not yet sent** · **not usable**. The window size is N.
- An ACK carries the seq # of the next byte expected from the other side, and is **cumulative**.
- **Q: How does the receiver handle out-of-order segments? A: The TCP spec doesn't say; it is up to the implementer.** In practice they are buffered.
- The **initial sequence number (ISN)** is chosen by each side, randomly rather than from 0. This reduces the chance that a segment from an old connection is mistaken for one in a new connection on the same 4-tuple, and it makes blind spoofing much harder (RFC 6528).

Simple telnet scenario [3-85]

Direction	Seq	ACK	Data	Meaning
A → B	42	79	'C'	The user types 'C'
B → A	79	**43**	'C'	B ACKs receipt of 'C' and echoes 'C' back. The ACK is **piggybacked** on data
A → B	43	**80**	—	A ACKs receipt of the echoed 'C'

The ACK number (43) is one more than the seq # (42) of the segment that triggered it, because 1 byte was received. Likewise, 80 = 79 + 1 [3-85 notes]. A pure ACK carries no data and consumes no sequence number: its seq is 43, and A's next data segment will also have seq 43.

6.4 Round-trip time estimation and timeout [3-86 ... 3-88]

- **Q: How should the TCP timeout be set?** Longer than the RTT, but the RTT varies. **Too short**: premature timeouts and unnecessary retransmissions. **Too long**: slow reaction to segment loss.
- **SampleRTT**: the time measured from a segment's transmission until its ACK arrives. **Retransmissions are ignored**. SampleRTT varies, so a "smoother" estimate averages several recent measurements. Most implementations take one sample per RTT, not one per segment.

```
EstimatedRTT = (1 − α)·EstimatedRTT + α·SampleRTT          α = 0.125 = 1/8
DevRTT       = (1 − β)·DevRTT + β·|SampleRTT − EstimatedRTT|   β = 0.25  = 1/4
TimeoutInterval = EstimatedRTT + 4·DevRTT
```

- This is an **exponential weighted moving average (EWMA)**: the influence of a past sample decreases exponentially fast. The slide graph shows RTTs from gaia.cs.umass.edu (Massachusetts) to fantasia.eurecom.fr (France), roughly 100–350 ms, with the smooth EstimatedRTT line through them.

- The **safety margin** is larger when variation is large [3-88 notes].
- The initial TimeoutInterval is **1 s** (RFC 6298). On a **timeout**, TCP **doubles** the interval (exponential backoff) rather than recomputing it. When the next ACK arrives for new data, the interval is recomputed from EstimatedRTT and DevRTT. The doubling is a crude form of congestion control.

> **BEYOND THE SLIDES · Karn's algorithm**
> When a segment has been retransmitted, its ACK is **ambiguous**: it may acknowledge the first copy or the second. Karn's rule: (1) take no SampleRTT from retransmitted segments, and (2) keep the backed-off timeout until an ACK arrives for a segment that was *not* retransmitted. The timestamps option, and QUIC's ever-increasing packet numbers, remove the ambiguity entirely.

6.5 TCP sender (simplified) [3-89, 3-150]

```
NextSeqNum = InitialSeqNum;  SendBase = InitialSeqNum
loop forever:
  event: data received from application above
      create segment with seq # NextSeqNum        (seq # = byte-stream # of first data
byte)
      if (timer not running) start timer          (think of the timer as for the oldest
unACKed segment)
      pass segment to IP;  NextSeqNum += length(data)
  event: timer timeout
      retransmit the not-yet-ACKed segment with the smallest seq #
      start timer                                  (expiration interval: TimeoutInterval)
  event: ACK received, with ACK field value y
      if (y > SendBase) {                          (ACKs previously unACKed segments)
          SendBase = y
          if (there are still unACKed segments) start timer }
      else { increment count of duplicate ACKs for y
             if (count == 3) resend segment with seq # y      ← fast retransmit }
```

6.6 TCP receiver: ACK generation [3-90] RFC 5681

Event at receiver	TCP receiver action
Arrival of an in-order segment with the expected seq #. All data up to the expected seq # is already ACKed	**Delayed ACK**: wait up to 500 ms for the next segment. If none arrives, send the ACK
Arrival of an in-order segment with the expected seq #. One other segment has an ACK pending	Immediately send a single cumulative ACK, acknowledging both in-order segments
Arrival of an out-of-order segment with a higher-than-expected seq #. A gap is detected	Immediately send a **duplicate ACK**, indicating the seq # of the next expected byte
Arrival of a segment that partially or completely fills the gap	Immediately send an ACK, provided the segment starts at the lower end of the gap

Delayed ACKs halve ACK traffic [3-90 notes]. RFC 1122 says at least every second full-size segment must be ACKed, and the delay must not exceed 500 ms. Linux uses about 40 ms and Windows about 200 ms.

6.7 Retransmission scenarios [3-91, 3-92]

Scenario	Timeline	Lesson
Lost ACK	A sends Seq = 92, 8 bytes. B's ACK = 100 is lost. A times out and resends Seq = 92, 8 bytes. B (already holding it) discards the duplicate and sends ACK = 100 again.	The receiver must re-ACK duplicates

Premature timeout	A sends Seq = 92 (8 B) and Seq = 100 (20 B). ACK = 100 and ACK = 120 are delayed, and the timer for 92 expires, so A resends Seq = 92. ACK 100 and then ACK 120 arrive, so SendBase = 120. B receives the duplicate 92 and sends a **cumulative ACK for 120**, not 100.	Only the oldest segment is resent (not GBN-style), and the cumulative ACK means segment 100 is not resent
Cumulative ACK covers an earlier lost ACK	A sends Seq = 92 (8 B) and Seq = 100 (20 B). ACK = 100 is lost, but ACK = 120 arrives before the timeout. A knows both arrived and sends Seq = 120, 15 bytes.	A cumulative ACK avoids a retransmission

6.8 TCP fast retransmit [3-93]

The timeout period is often relatively long, which causes a long delay before a lost packet is resent. **If the sender receives 3 additional ACKs for the same data (“triple duplicate ACKs”), it resends the unACKed segment with the smallest seq #**, since that segment was likely lost, so it does not wait for the timeout.

Slide example: A sends Seq 92 (8 B), Seq 100 (20 B, *lost*), then three more segments. B sends ACK 100 (for the first) and then ACK 100 three more times as each later segment arrives. After receiving the triple duplicate ACK, A resends Seq = 100, 20 bytes, before the timer expires. The original ACK does not count as a duplicate: it takes **4 ACKs with the same number in total**.

6.9 Is TCP Go-Back-N or Selective Repeat?

It is a hybrid. Like **GBN**, it uses **cumulative ACKs** and a **single timer**, and the receiver only reports the next expected byte. Like **SR**, the receiver **buffers** out-of-order segments, and on a timeout the sender resends **only one segment** (the oldest unACKed), not the whole window. With the **SACK** option (RFC 2018), TCP’s selective acknowledgement makes it very close to SR.

> **BEYOND THE SLIDES · Other TCP mechanisms that come up in exams**
>
> - **Nagle’s algorithm** (RFC 896): while there is unACKed data outstanding, buffer small writes and send them as one segment when the ACK returns or a full MSS has accumulated. This prevents the “tinygram” problem (e.g. 41-byte packets carrying a 1-byte keystroke). Interactive applications disable it with `TCP_NODELAY`. Nagle combined with delayed ACK can add up to 200–500 ms of latency.
> - **Silly window syndrome**: a slow receiver advertises tiny windows and the sender fills each one with a tiny segment. **Clark’s solution** (receiver side): do not advertise a window until min(MSS, ½ buffer) is free. Nagle is the sender-side fix.
> - **RST**: sent when a segment arrives for a port with no listening socket (TCP’s version of “port unreachable”) or to abort a connection abruptly. It needs no ACK, and data in the buffers is discarded.
> - **Keepalive**: an optional probe on an idle connection, by default after 2 hours, to detect a crashed peer.
> - **Half-close**: `shutdown(SHUT_WR)` sends a FIN while the socket can still receive, because each direction is closed independently.

7 · TCP flow control [3-95 … 3-100]

The question [3-95]: what happens if the network layer delivers data faster than the application layer removes it from the socket buffers? Segments come up the stack, and their payloads are written *into* the socket buffer. The application takes data *out* by performing socket reads. If the application is slow (busy, or reading infrequently), the buffer fills and new data would be dropped.

Flow control: the receiver controls the sender, so that the sender will not overflow the receiver's buffer by transmitting too much, too fast. It is a **speed-matching** service.

```
rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)          ← free buffer space, advertised
in every segment
Sender:  LastByteSent − LastByteAcked ≤ rwnd                ← limits "in-flight" data
Combined with congestion control:  LastByteSent − LastByteAcked ≤ min(cwnd, rwnd)
```

- The TCP receiver "advertises" its free buffer space in the **rwnd** field of the TCP header. RcvBuffer is set through socket options (a typical default is **4096 bytes**), and many operating systems **auto-adjust** RcvBuffer.
- The sender limits the amount of unACKed ("in-flight") data to the received rwnd, which **guarantees that the receive buffer will not overflow**.
- The value changes over time as the application reads and data arrives [3-99 notes]. Because TCP is full duplex, **each side** keeps its own receive buffer and advertises its own rwnd.

7.1 The zero-window problem

Suppose B's buffer fills, so rwnd = 0, and A stops sending. B has nothing to send to A, so when its application empties the buffer, no segment carries the new rwnd to A, and A is blocked forever. **Fix**: while rwnd = 0, A keeps sending **1-byte zero-window probe** segments, driven by the **persist timer** with exponential backoff. B's ACKs to these probes eventually carry a non-zero rwnd.

7.2 Flow control vs congestion control

	Flow control	Congestion control
Problem	A fast sender overwhelms a slow **receiver**	Too many senders overwhelm the **network** (routers and links)
Variable	rwnd: explicit, carried in the header	cwnd: private to the sender, inferred from loss or delay, or signalled by ECN
Who decides	The receiver	The sender, reacting to the network
Symptom if missing	Receive-buffer overflow, and data lost at the host	Router queue overflow, long delays, congestion collapse

> **BEYOND THE SLIDES · Window-limited throughput and window scaling**
>
> Throughput ≤ rwnd / RTT. With the maximum 16-bit window (65,535 B) and RTT = 100 ms, that is ≈ 5.2 Mbps, *whatever the link speed*. To keep a pipe full, rwnd must be at least the **bandwidth-delay product** (R × RTT). For 1 Gbps × 100 ms that is 12.5 MB, which needs the **window scale** option. Receive-buffer auto-tuning in Linux and Windows grows RcvBuffer toward the BDP automatically.

8 · TCP connection management [3-101 … 3-108, 3-151, 3-152]

Before exchanging data, the sender and receiver "handshake" [3-101]: they **agree to establish a connection** (each knowing the other is willing), and they **agree on connection parameters** (starting seq #s, receive buffer sizes). The resulting **connection state** is ESTAB, with **connection variables**: seq

client-to-server, seq # server-to-client, and the rcvBuffer size at the server and at the client. In Java: `Socket clientSocket = new Socket("hostname","port number");` and `Socket connectionSocket = welcomeSocket.accept();`

8.1 Will a 2-way handshake always work? [3-102 ... 3-105]

2-way handshake: the client chooses x and sends `req_conn(x)`; the server replies `acc_conn(x)`; both enter ESTAB. Like Alice saying "Let's talk" and Bob saying "OK". In a network, though, there are **variable delays**, **retransmitted messages** (e.g. req_conn(x) resent after a loss), **message reordering**, and **neither side can "see" the other**.

Scenario	What happens
1. No problem	req_conn(x) → acc_conn(x). The client sends data(x+1), the server accepts it and ACKs (x+1). The connection completes and both close.
2. **Half-open connection** (no client)	The client sends req_conn(x), its timer expires, and it **retransmits** req_conn(x). acc_conn(x) arrives and the client enters ESTAB, finishes, and terminates. The **delayed retransmitted req_conn(x)** then reaches the server, which enters ESTAB and allocates resources for a client that **no longer exists**.
3. **Duplicate data accepted**	As in scenario 2, but the client also retransmitted data(x+1). After the client terminates, the old req_conn(x) opens a new connection at the server, and the old data(x+1) is **accepted as new data**. This corrupts the application, e.g. by processing a payment twice.

8.2 TCP 3-way handshake [3-106, 3-151]

Step	Segment	Client state	Server state
0	The server has called socket, bind, listen and accept (blocking). The client has called socket	CLOSED	LISTEN
1	**SYN**: the client chooses its initial seq # x and sends a TCP SYN segment: SYNbit = 1, Seq = x. No data	SYN_SENT	LISTEN
2	**SYNACK**: the server chooses its initial seq # y and sends SYNbit = 1, Seq = y, ACKbit = 1, ACKnum = x + 1. It allocates buffers and creates a **new socket** for this client	SYN_SENT	SYN_RCVD
3	**ACK**: receiving the SYNACK tells the client the server is live. The client sends ACKbit = 1, ACKnum = y + 1 (Seq = x + 1). **This segment may contain client-to-server data**	ESTAB	SYN_RCVD
4	Receiving the ACK(y) tells the server the client is live	ESTAB	ESTAB

- SYN and FIN each **consume one sequence number**, even though they carry no data. That is why the ACKs are x + 1 and y + 1, and why the first data byte has seq x + 1.
- Python socket calls on the slide: client `clientSocket = socket(AF_INET, SOCK_STREAM)`, `clientSocket.connect((serverName, serverPort))`. Server `serverSocket = socket(AF_INET, SOCK_STREAM)`, `serverSocket.bind(('', serverPort))`, `serverSocket.listen(1)`, `connectionSocket, addr = serverSocket.accept()`.
- **Human 3-way handshake** [3-107], from rock climbing: "**On belay?**" (are you ready with my safety rope?) → "**Belay on.**" → "**Climbing.**"
- **Why three messages fix the 2-way problems**: the server does not consider the connection established until the client ACKs the server's *fresh* ISN y. An old duplicate SYN gets a SYNACK carrying a new y, and the client (which is not expecting a connection) either ignores it or answers with RST. No half-open connection survives, and no stale data is accepted.

8.3 Closing a TCP connection [3-108, 3-152]

Step	Segment	Client	Server
1	The client calls close() and sends **FINbit = 1, seq = x**. It can no longer send, but it can still receive data	FIN_WAIT_1	ESTAB
2	The server sends **ACKbit = 1, ACKnum = x + 1**. It can still send data (half-closed)	FIN_WAIT_2 (waits for the server to close)	CLOSE_WAIT
3	The server calls close() and sends **FINbit = 1, seq = y**. It can no longer send data	FIN_WAIT_2	LAST_ACK
4	The client sends **ACKbit = 1, ACKnum = y + 1**	**TIMED_WAIT**: waits 2 × maximum segment lifetime, then CLOSED	CLOSED on receiving the ACK

- The client and server **each close their own side** of the connection by sending a TCP segment with FIN bit = 1, and respond to a received FIN with an ACK. On receiving a FIN, the ACK can be **combined with the receiver's own FIN** (3 segments instead of 4), and **simultaneous FIN exchanges** can be handled.
- **TIME_WAIT (2·MSL)**, typically 30 s, 1 min or 2 min: (a) if the final ACK is lost, the server retransmits its FIN and the client must still be there to re-ACK it; (b) delayed duplicates of this connection must expire before the same 4-tuple can be reused. This is why restarting a server immediately can give "Address already in use" unless `SO_REUSEADDR` is set.

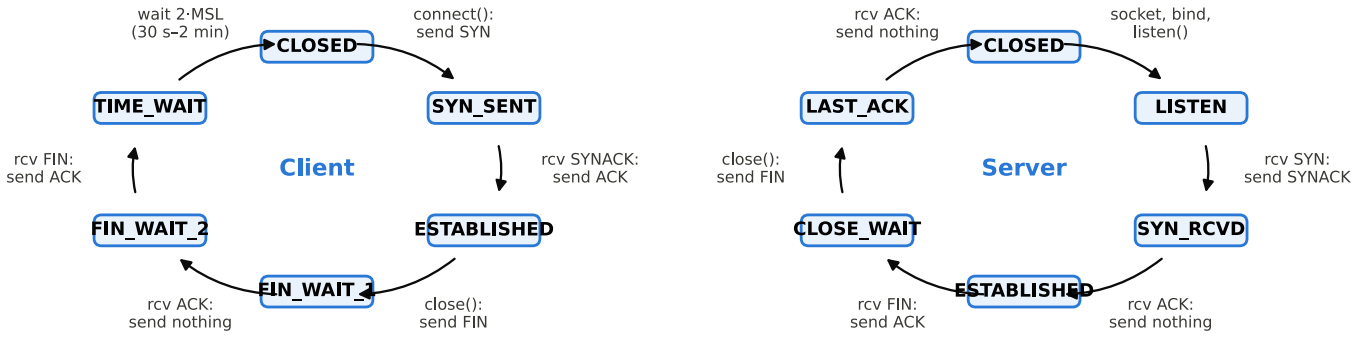


Figure 4. Typical TCP state sequences for a client (active open and active close) and a server (passive open and passive close).

BEYOND THE SLIDES · Edge cases

- **SYN to a closed port**: the host replies with **RST**. A UDP datagram to a closed port triggers an **ICMP port unreachable**. Port scanners exploit both.
- **Simultaneous open**: both sides send a SYN at the same time. Each goes SYN_SENT → SYN_RCVD → ESTAB, which takes 4 segments. It is rare but legal.
- **SYN retransmission**: if the SYNACK does not arrive, the client retransmits the SYN after about 1 s, then 2 s, 4 s, and so on.
- **SYN flood**: many SYNs with no final ACK fill the half-open (SYN) queue. The defence is SYN cookies (§13).

9 · Principles of congestion control [3-110 ... 3-122]

Congestion, informally, is “too many sources sending too much data too fast for the *network* to handle”. It shows up as **long delays** (queueing in router buffers) and **packet loss** (buffer overflow at routers). It is **different from flow control**, and it is a “top-10 problem” in networking [3-110].

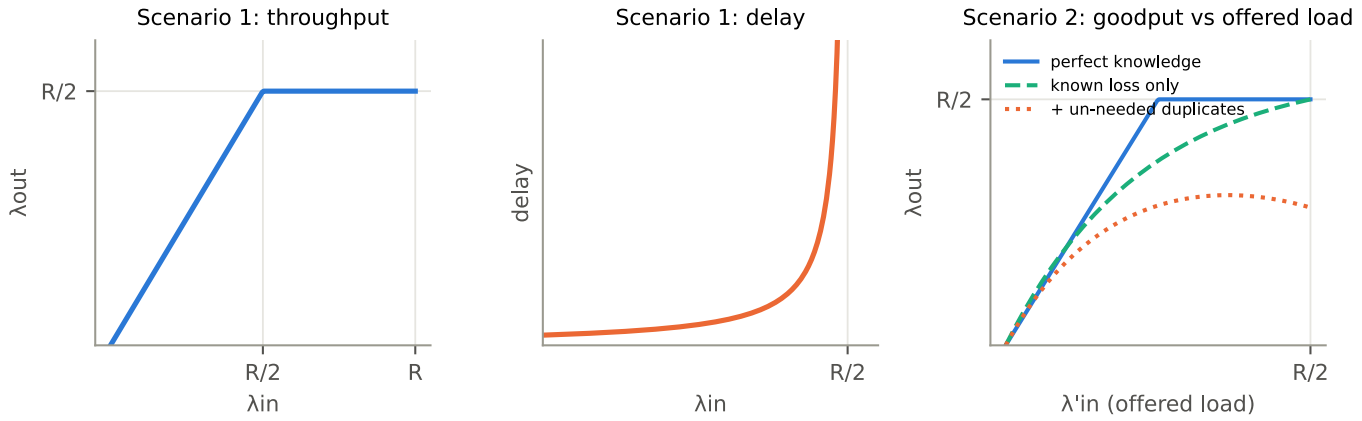


Figure 5. Left and middle: scenario 1, where throughput caps at R/2 and delay grows without bound as λin → R/2. Right: scenario 2, where goodput falls further below the ideal as the sender’s knowledge becomes more realistic.

Notation: λ_{in} is the rate at which the application sends original data. λ'_{in} is the **offered load**: original data *plus retransmissions*, so $\lambda'_{in} \geq \lambda_{in}$. λ_{out} is the rate delivered to the receiving application (the **goodput**).

9.1 Scenario 1: two flows, one router, infinite buffers [3-111]

The input and output links have capacity R, and there are no retransmissions because nothing is ever lost. **Maximum per-connection throughput is R/2**. As the arrival rate λ_{in} approaches R/2, **delays become large** and grow without bound (the queue grows forever). **Cost revealed**: large queueing delays as the arrival rate nears link capacity, even with infinite buffers.

9.2 Scenario 2: one router, finite buffers [3-112 ... 3-117]

The sender retransmits lost or timed-out packets. The application-layer input equals the application-layer output ($\lambda_{in} = \lambda_{out}$), but the transport-layer input includes retransmissions ($\lambda'_{in} \geq \lambda_{in}$).

Case	Assumption	Result
Idealisation: **perfect knowledge**	The sender sends only when router buffer space is available, so nothing is lost	$\lambda_{out} = \lambda'_{in}$ up to R/2 (the blue line)
Idealisation: **some perfect knowledge**	Packets can be dropped at a full buffer, and the sender knows exactly when a packet has been dropped and resends only packets known to be lost	Some capacity carries retransmissions, so goodput is below R/2. The textbook example: at $\lambda'_{in} = R/2$, goodput ≈ R/3 (0.333R original and 0.166R retransmitted)
Realistic: un-needed duplicates	Packets can be lost and must be retransmitted, but the sender can also time out **prematurely** and send two copies, **both of which are delivered**	The link carries useless copies, so goodput falls further. If every packet is forwarded twice on average, goodput approaches R/4

“Costs” of congestion [3-117]: **more work** (retransmission) for a given receiver throughput; **unneeded retransmissions**, where the link carries multiple copies of a packet; and a **decreasing maximum achievable throughput**.

9.3 Scenario 3: four senders, multi-hop paths [3-118, 3-119]

Four senders use 2-hop paths that overlap, with finite shared output-link buffers and timeout/retransmit. **Q: What happens as λ_{in} and λ'_{in} increase? A:** as the red flow's λ'_{in} increases, **all arriving blue packets at the upper queue are dropped**, so the blue throughput → 0. At high load, end-to-end throughput **collapses toward zero**, which is **congestion collapse**.

Another cost: when a packet is dropped, **any upstream transmission capacity and buffering used for that packet was wasted**.

9.4 Insights [3-120]

1. Throughput can never exceed capacity.
2. Delay increases as capacity is approached.
3. Loss and retransmission decrease effective throughput.
4. Un-needed duplicates decrease effective throughput further.
5. Upstream transmission capacity and buffering are wasted on packets lost downstream.

9.5 Approaches to congestion control [3-121, 3-122]

	End-end congestion control	**Network-assisted congestion control**
Feedback	None explicit from the network. Congestion is **inferred** from observed loss and delay	Routers provide **direct feedback** to the sending and/or receiving hosts whose flows pass through the congested router
What is signalled	—	A congestion indication (e.g. a single bit), a congestion level, or an explicit sending rate
Examples	**TCP** (classic Tahoe/Reno/CUBIC; delay-based Vegas/BBR)	**TCP ECN**, ATM ABR, DECbit

Network feedback can arrive in two ways: a **direct choke packet** from the router to the sender, or **marking** a packet on its way to the receiver, which then tells the sender. Marking takes at least a full RTT to reach the sender.

10 · TCP congestion control [3-124 … 3-139, 3-153, 3-154]

10.1 AIMD [3-124, 3-125]

Approach: senders increase their sending rate until packet loss (congestion) occurs, then decrease the rate on a loss event.

- **Additive Increase**: increase the sending rate by **1 maximum segment size every RTT** until loss is detected.
- **Multiplicative Decrease**: cut the sending rate **in half** at each loss event. In detail, the rate is **cut in half on loss detected by triple duplicate ACK (TCP Reno)** and **cut to 1 MSS on loss detected by timeout (TCP Tahoe)**.
- The result is the **AIMD sawtooth**, which **probes for bandwidth**.
- **Why AIMD?** AIMD, a *distributed, asynchronous* algorithm, has been shown to **optimise congested flow rates network-wide** and to have **desirable stability properties**. Chiu and Jain (1989) showed that of the four linear policies (AIAD, AIMD, MIAD, MIMD), only AIMD converges to both efficiency and fairness.

10.2 Details [3-126]

```
LastByteSent − LastByteAcked ≤ cwnd            (and ≤ rwnd)
TCP sending behaviour: send cwnd bytes, wait one RTT for ACKs, then send more
TCP rate ≈ cwnd / RTT   bytes/sec
```

cwnd is **dynamically adjusted** in response to observed network congestion; this is how TCP congestion control is implemented. **ACK clocking** (self-clocking): new segments are released as ACKs return, so the sending rate automatically matches the rate at which the bottleneck delivers packets.

10.3 Slow start [3-127]

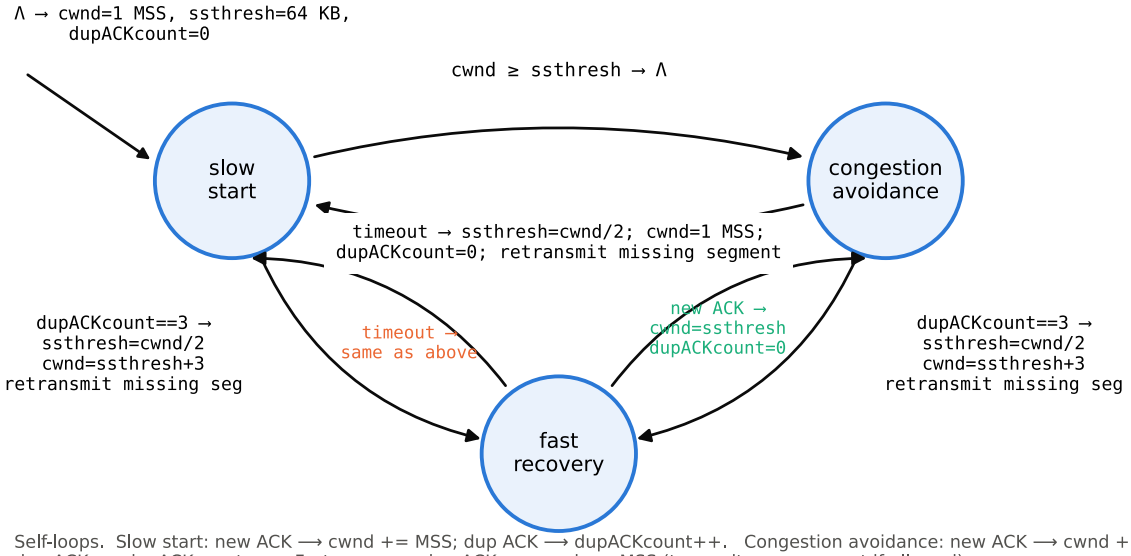
- When the connection begins, increase the rate **exponentially** until the first loss event. Initially **cwnd = 1 MSS**, and cwnd **doubles every RTT**. This is done by **incrementing cwnd by 1 MSS for every ACK received**. Host A sends one segment, then two, then four.
- Summary: the **initial rate is slow, but it ramps up exponentially fast**. Example: MSS = 500 B and RTT = 200 ms give an initial rate of only 20 kbps.
- Modern stacks start with an initial window of **10 MSS** (RFC 6928), but the doubling logic is the same.

10.4 From slow start to congestion avoidance [3-128]

Q: When should the exponential increase switch to linear? A: When cwnd reaches 1/2 of its value before the timeout. This is implemented with a variable **ssthresh**. On a loss event, **ssthresh is set to 1/2 of cwnd just before the loss**. In congestion avoidance, cwnd grows by MSS × (MSS/cwnd) per ACK, which is about 1 MSS per RTT.

The slide 3-128 graph (K&R Fig. 3.53): ssthresh is initially 8 MSS. Slow start runs 1, 2, 4, 8, then linear 9, 10, 11, 12. A loss by triple duplicate ACK at cwnd = 12 sets ssthresh = 6. **TCP Tahoe** drops to cwnd = 1 and slow-starts again up to 6. **TCP Reno** drops to **cwnd = ssthresh + 3 = 9** and grows linearly. Many courses simplify Reno to cwnd = 6. State your convention.

10.5 The complete TCP Reno congestion-control FSM [3-129]



Self-loops. Slow start: new ACK ⟶ cwnd += MSS; dup ACK ⟶ dupACKcount++. Congestion avoidance: new ACK ⟶ cwnd += MSS·(MSS/cwnd); dup ACK ⟶ dupACKcount++. Fast recovery: dup ACK ⟶ cwnd += MSS (transmit new segment if allowed).

Figure 6. TCP congestion control FSM (Reno). Tahoe is the same without the fast-recovery state: on 3 duplicate ACKs it behaves as on a timeout, except that it performs a fast retransmit.

State	Event	Action	Next state
(start)	Λ	`cwnd = 1 MSS; ssthresh = 64 KB; dupACKcount = 0`	Slow start
Slow start	`new ACK`	`cwnd = cwnd + MSS; dupACKcount = 0; transmit new segment(s) as allowed`	Slow start
Slow start	`cwnd ≥ ssthresh`	Λ	Congestion avoidance
Slow start / CA	`duplicate ACK`	`dupACKcount++`	Same
Slow start / CA	`dupACKcount == 3`	`ssthresh = cwnd/2; cwnd = ssthresh + 3·MSS; retransmit missing segment`	Fast recovery
Any state	`timeout`	`ssthresh = cwnd/2; cwnd = 1 MSS; dupACKcount = 0; retransmit missing segment`	Slow start
Congestion avoidance	`new ACK`	`cwnd = cwnd + MSS·(MSS/cwnd); dupACKcount = 0; transmit new segment(s)`	Congestion avoidance
Fast recovery	`duplicate ACK`	`cwnd = cwnd + MSS (“window inflation”: each dup ACK means a packet left the network); transmit new segment(s)`	Fast recovery
Fast recovery	`new ACK`	`cwnd = ssthresh (“deflate”); dupACKcount = 0`	Congestion avoidance

BEYOND THE SLIDES · NewReno and SACK recovery

Classic Reno leaves fast recovery on the first new ACK. If several segments from one window were lost, it may then need a timeout for the second one. **NewReno** (RFC 6582) treats a *partial ACK* (one that acknowledges some but not all of the data outstanding at the time of loss) as a sign of another loss: it retransmits the next hole immediately and stays in fast recovery. **SACK-based recovery** (RFC 6675) knows exactly which holes exist. Both are standard today.

10.6 Average TCP throughput and long, fat pipes [3-153, 3-154]

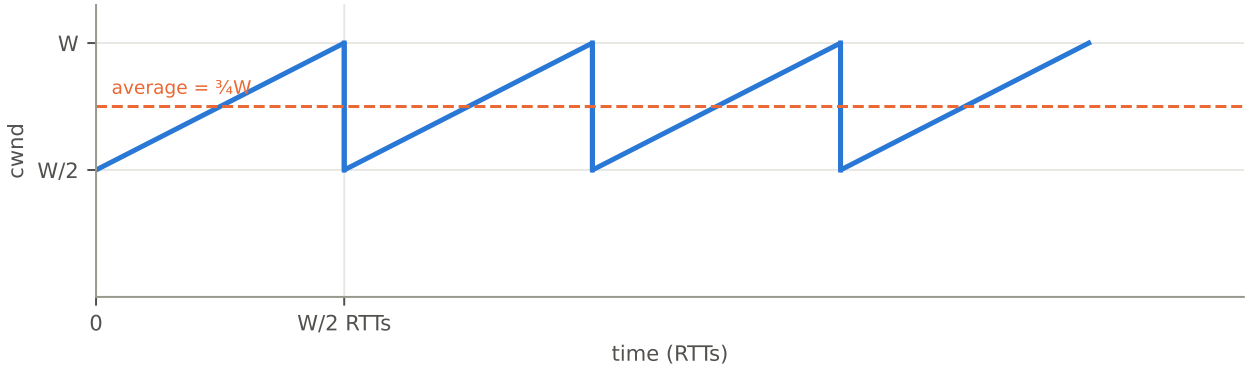


Figure 7. Idealised AIMD sawtooth, ignoring slow start and assuming there is always data to send. The window oscillates between W/2 and W, so the average is ¾W.

```
avg TCP throughput = (3/4) · W / RTT   bytes/sec      (W = window size in bytes at which
loss occurs)

Mathis et al. (1997), in terms of segment loss probability L:
  one sawtooth cycle lasts W/2 RTTs and carries  (W/2 + W)/2 · W/2 = (3/8)W²  segments
  one loss per cycle  ⇒  L = 1 / ((3/8)W²)  ⇒  W = √(8 / (3L))
  throughput = (3/4)·W·MSS / RTT = (3/4)·√(8/3)·MSS / (RTT·√L)  ≈  1.22 · MSS / (RTT · √L)
```

Long, fat pipe example [3-154]: 1500-byte segments, 100 ms RTT, and a target of 10 Gbps throughput require **W = 83,333 in-flight segments**. The formula then requires a loss rate of **L = $2 \cdot 10^{-10}$**, which is a very small loss rate. Hence **new versions of TCP for long, high-speed scenarios** are needed.

Consequences of the formula: throughput ∝ 1/RTT (flows with a short RTT win: **RTT unfairness**) and ∝ 1/√L (you must quarter the loss rate to double throughput).

10.7 TCP CUBIC [3-130, 3-131]

- **Is there a better way than AIMD to "probe" for usable bandwidth?**
- **Insight**: W_{max} is the sending rate at which congestion loss was detected. The congestion state of the bottleneck link probably (?) hasn't changed much. So after cutting the rate or window in half on a loss, **initially ramp up to W_{max} faster, but then approach W_{max} more slowly**.
- **K**: the point in time at which the TCP window size will reach W_{max}. K itself is tunable.
- W is increased as a function of the **cube of the distance between the current time and K**: larger increases when further from K, and smaller, more cautious increases when nearer K.
- CUBIC is the **default in Linux** and the most popular TCP on popular web servers.

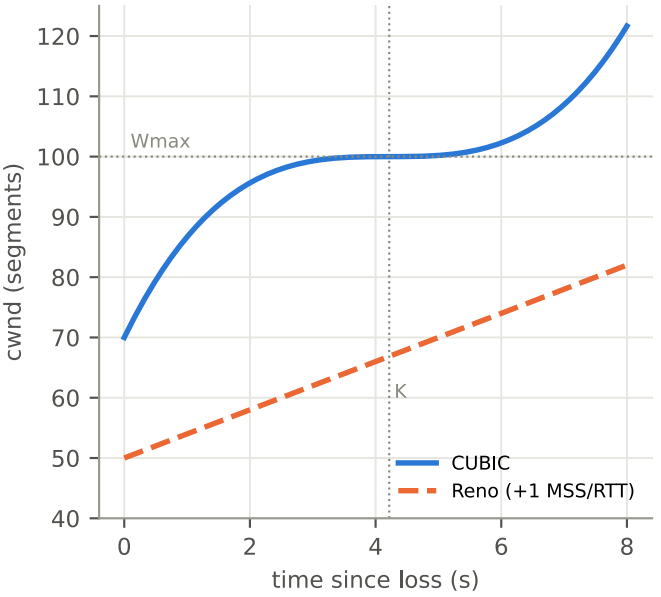


Figure 8. CUBIC (W_{max} = 100, C = 0.4, β = 0.3) vs Reno after the same loss.

```
W(t) = C·(t − K)³ + W_max        K = ∛(W_max·β / C)        (RFC 9438: C = 0.4, β = 0.3)
After loss: cwnd = (1 − β)·W_max = 0.7·W_max  (CUBIC cuts by 30%, not 50%)
t = time since the last loss (wall-clock, not RTTs), so growth is RTT-independent
```

The curve is concave (fast, then flattening) before K and convex (slow, then accelerating) after K. The plateau near W_{max} keeps the queue stable, and the convex part probes for newly available bandwidth. Because growth depends on time rather than RTT, flows with different RTTs share more fairly than under Reno.

10.8 The bottleneck link and delay-based congestion control [3-132 … 3-135]

- Classic TCP and CUBIC increase the sending rate until packet loss occurs at some router's output: the **bottleneck link**, which is almost always busy.
- Understanding congestion means focusing on the congested bottleneck. Increasing the sending rate **will not increase end-to-end throughput** once the bottleneck is congested; it only **increases the measured RTT** (the queue grows). **Goal: "keep the end-end pipe just full, but not fuller".**
- **Delay-based approach**: RTT_{min} is the minimum observed RTT (the uncongested path). The uncongested throughput with window cwnd is **cwnd/RTT_{min}**. Measured throughput = bytes sent in the last RTT interval / $RTT_{measured}$.

  ```
  if measured throughput "very close" to uncongested throughput: increase cwnd linearly /*
  path not congested */
  else if measured throughput "far below" uncongested throughput: decrease cwnd linearly /*
  path congested */
  ```

- This achieves congestion control **without inducing or forcing loss**, maximising throughput ("keeping the pipe just full…") while keeping delay low ("…but not fuller"). A number of deployed TCPs take a delay-based approach. **BBR** is deployed on Google's internal backbone network.

> **BEYOND THE SLIDES · Vegas and BBR in one paragraph each**
>
> **TCP Vegas** (1994): Diff = (cwnd/RTT_{min} − cwnd/RTT) × RTT_{min} estimates the number of *extra packets queued*. If Diff < α (e.g. 1), cwnd++; if Diff > β (e.g. 3), cwnd−−. It detects congestion before loss, but it yields to loss-based flows, which keep filling the queue that Vegas is trying to keep empty.
>
> **BBR** (Bottleneck Bandwidth and Round-trip propagation time, Google 2016): it continuously estimates **BtlBw** (maximum delivery rate) and **RTprop** (minimum RTT). It paces packets at ≈ BtlBw and caps in-flight data at ≈ BtlBw × RTprop (the BDP), probing periodically with a pacing gain of 1.25 and then 0.75, and occasionally draining the queue to re-measure RTprop. It is model-based rather than loss-driven, so random loss (as on wireless links) does not cut its rate.

10.9 Explicit Congestion Notification (ECN) [3-136]

- TCP deployments often implement **network-assisted** congestion control. **Two bits in the IP header (ToS field)** are marked by a network router to indicate congestion. The **marking policy is chosen by the network operator**.
- The congestion indication is carried to the destination, which **sets the ECE bit on the ACK segment** to notify the sender. This involves both IP (ECN bit marking in the IP header) and TCP (C and E bits in the TCP header).

IP ECN bits	Meaning
00	Not ECN-capable transport
10 / 01	ECT(0) / ECT(1): the endpoints support ECN (negotiated during the SYN handshake)
11	**CE: Congestion Experienced**, set by a router instead of dropping the packet

Flow of events: router sets CE → receiver sets **ECE** on its ACKs → sender halves cwnd (as for a loss, but **nothing was lost**, so there is no retransmission) and sets **CWR** → receiver stops echoing. Routers usually decide when to mark using an **AQM** scheme such as RED.

BEYOND THE SLIDES · Active Queue Management (AQM) and bufferbloat

- **Drop-tail** queues drop only when full. Loss-based TCP keeps them full, so large buffers cause huge standing delays: **bufferbloat** (seconds of lag on home routers).
- **RED** (Random Early Detection): drop or mark with a probability that rises with the *average* queue length between $\min_{th}$ and $\max_{th}$. This signals senders early and at random, avoiding the synchronised backoff of many flows.
- **CoDel**: drop or mark when the *queueing delay* (sojourn time) stays above about 5 ms for about 100 ms.
- **Buffer sizing**: the classic rule is buffer = RTT × C, which lets one AIMD flow keep the link 100% busy through its halving. With N desynchronised flows, RTT × C / √N suffices.

10.10 TCP fairness [3-137 ... 3-139]

Fairness goal: if K TCP sessions share the same bottleneck link of bandwidth R, each should have an average rate of **R/K**.

Q: Is TCP fair? Take two competing TCP sessions. **Additive increase** gives a slope of 1 as throughput increases, and **multiplicative decrease** reduces throughput proportionally. Each loss halves both rates and moves the operating point toward the equal-share line, so the pair converges to R/2 each.

A: Yes, under idealised assumptions: the same RTT, and a fixed number of sessions, all in congestion avoidance.

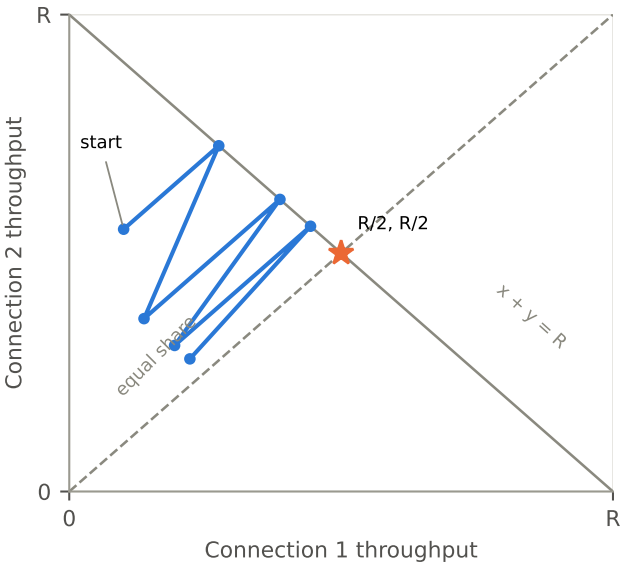


Figure 9. AIMD phase plot: each increase-and-halve cycle halves the gap between the two flows.

Must all network apps be “fair”? [3-139]

- **Fairness and UDP**: multimedia apps often do not use TCP, because they do not want their rate throttled by congestion control. Instead they use UDP, sending audio or video at a constant rate and tolerating packet loss. **There is no “Internet police” policing the use of congestion control.**
- **Fairness and parallel TCP connections**: an application can open multiple parallel connections between two hosts, and web browsers do this. On a link of rate R with 9 existing connections, a new app that asks for 1 TCP connection gets R/10, but a new app that asks for 11 TCP connections gets R/2.

BEYOND THE SLIDES · More on fairness

- **RTT unfairness**: throughput ∝ 1/RTT, so a flow with half the RTT gets about twice the bandwidth under Reno. CUBIC reduces this.
- **AIAD does not converge**: additive increase and additive decrease keep the gap between two flows constant forever. **MIMD** also preserves the *ratio*. Only AIMD shrinks the gap, which is the Chiu–Jain result.
- **TCP-friendliness**: a non-TCP flow is "TCP-friendly" if its long-term rate does not exceed that of a TCP flow under the same conditions. **TFRC** (RFC 5348) sets a UDP media flow's rate using the Mathis-style equation, giving smooth rates with TCP-fair averages.
- **Background traffic**: LEDBAT (used by BitTorrent's µTP) is delay-based and deliberately *less* aggressive than TCP, backing off as soon as it sees queueing delay. This links back to the BitTorrent questions in your last paper.

11 · Evolution of transport-layer functionality [3-141 ... 3-145]

TCP and UDP have been the principal transport protocols for 40 years. Different "flavours" of TCP have been developed for specific scenarios [3-141]:

Scenario	Challenges	Typical answer
Long, fat pipes (large data transfers)	Many packets "in flight"; a loss shuts down the pipeline	CUBIC, BBR, HighSpeed TCP, window scaling, SACK
Wireless networks	Loss due to noisy wireless links and mobility; **TCP treats this as congestion loss**	Link-layer retransmission (802.11 ARQ), split TCP, TCP Westwood, BBR
Long-delay links	Extremely long RTTs (satellite, about 600 ms for GEO)	Performance-enhancing proxies, larger initial windows
Data center networks	Latency-sensitive; incast (many-to-one bursts)	**DCTCP**: cuts cwnd in proportion to the fraction of ECN-marked packets
Background traffic flows	Low-priority "background" TCP flows	LEDBAT

The other trend is **moving transport-layer functions to the application layer, on top of UDP**, as in HTTP/3 over QUIC.

11.1 QUIC: Quick UDP Internet Connections [3-142 ... 3-145] RFC 9000/9001/9002, HTTP/3 = RFC 9114

- An **application-layer protocol, on top of UDP**, designed to **increase the performance of HTTP**. It is deployed on many Google servers and apps (Chrome, the mobile YouTube app). Stack: **HTTP/3 → QUIC → UDP → IP**, in place of HTTP/2 → TLS → TCP → IP.
- It **adopts the approaches studied in this chapter** for connection establishment, error control and congestion control. **Error and congestion control**: "Readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones" (from the QUIC specification).
- **Connection establishment**: reliability, congestion control, authentication, encryption and state are established in **one RTT**. TCP + TLS need **2 serial handshakes**: the TCP handshake for reliability and congestion-control state, then the TLS handshake for authentication and crypto state [3-144].
- **Multiple application-level "streams"** are multiplexed over a single QUIC connection, with **separate reliable data transfer and security** per stream and **common congestion control**.
- **No HOL blocking** [3-145]: (a) with HTTP/1.1 (or HTTP/2) over TLS over TCP, an error or loss in the single byte stream stalls every object behind it. (b) With HTTP over QUIC, a loss in one stream stalls only that stream.

BEYOND THE SLIDES · QUIC details worth knowing

- **Connection IDs** instead of the 4-tuple identify a connection, enabling **connection migration** (a phone moving from Wi-Fi to 4G keeps its connection) and helping with NAT rebinding.
- **Packet numbers are never reused**: a retransmission carries the same *data* in a *new* packet number, so there is no retransmission ambiguity (Karn's problem disappears) and RTT samples are always valid. ACK frames list ranges (like a built-in SACK) and report the ACK delay.
- **Built-in TLS 1.3**: all payloads and most header fields (including packet numbers) are encrypted. This improves privacy and prevents middlebox ossification, but passive tools like Wireshark cannot see transport details without the keys.
- **0-RTT resumption**: a returning client can send data in its very first flight, at the risk of **replay attacks**, so 0-RTT data should be idempotent (e.g. GET requests).
- **Anti-amplification**: before the client's address is validated, a server may send at most **3×** the bytes it has received, which blocks UDP reflection attacks.
- Runs in **user space**, so new congestion controllers (e.g. BBR) ship with an app update rather than an OS kernel update.

BEYOND THE SLIDES · Other transport protocols (possible 2-mark "name one" questions)

SCTP	Message-oriented and reliable, with **multi-streaming** (no HOL blocking between streams) and **multi-homing** (several IP addresses per endpoint). Used in telecom signalling (SS7 over IP) and in WebRTC data channels (over DTLS).
DCCP	Unreliable datagrams *with* congestion control: UDP plus congestion control.
MPTCP	Multipath TCP: one connection striped over several paths (Wi-Fi and cellular), with coupled congestion control. Used by Apple's Siri.
RTP/RTCP	Carried over UDP for real-time media: sequence numbers, timestamps, payload type. RTCP gives loss and jitter feedback.

12 · Sockets and tools: the transport layer as programmers and analysts see it

12.1 The socket API mapped to transport-layer events

UDP (from Chapter 2)

```
# client
s = socket(AF_INET, SOCK_DGRAM)
s.sendto(msg.encode(), (serverName, 12000))
reply, serverAddr = s.recvfrom(2048)

# server
s = socket(AF_INET, SOCK_DGRAM)
s.bind(('', 12000))
while True:
    msg, clientAddr = s.recvfrom(2048)
    s.sendto(msg.upper(), clientAddr)
```

There is no connection: every sendto() names the destination, and recvfrom() returns the sender's (IP, port) as the "return address". One server socket serves every client.

TCP (from Chapter 2)

```
# client
s = socket(AF_INET, SOCK_STREAM)
s.connect((serverName, 12000))   # SYN…
handshake
s.send(msg.encode())
reply = s.recv(1024)
s.close()                        # FIN

# server
ws = socket(AF_INET, SOCK_STREAM)
ws.bind(('', 12000)); ws.listen(1)
while True:
    cs, addr = ws.accept()       # new socket
    msg = cs.recv(1024); cs.send(msg.upper())
    cs.close()
```

Call	What TCP does underneath	Blocks until…
`listen(backlog)`	Moves the socket to LISTEN. The kernel will now complete handshakes on its own	Does not block
`connect()`	Sends a SYN (SYN_SENT), receives the SYNACK and sends the ACK	The handshake completes (or fails with a timeout or RST: "connection refused")
`accept()`	Takes an **already-established** connection off the accept queue and returns a new socket bound to its 4-tuple	A completed connection is available
`send()`	Copies bytes into the **send buffer**. TCP transmits them when cwnd, rwnd and Nagle allow. Returning does *not* mean the peer received the data	There is room in the send buffer. A full buffer means flow or congestion control is pushing back on the application
`recv()`	Copies bytes out of the receive buffer, which increases rwnd	At least 1 byte is available. **Returns 0 bytes** once the peer's FIN has arrived (end of file)
`close()`	Sends a FIN (FIN_WAIT_1). If unread data remains in the receive buffer, it sends RST instead	Normally returns immediately; SO_LINGER changes this

12.2 Blocking vs non-blocking I/O

- **Blocking** (the default): the call suspends the thread until it can complete. It is simple, but one thread handles one connection, so a server needs a thread (or process) per client.
- **Non-blocking** (`s.setblocking(False)` or O_NONBLOCK): a call that cannot complete immediately returns an error (EAGAIN / EWOULDBLOCK, or EINPROGRESS for connect()). It is combined with **I/O multiplexing** (select, poll, epoll or kqueue; Python's selectors and asyncio) so that one thread can serve thousands of connections. This is how nginx and Node.js work.
- **Timeouts**: `s.settimeout(2.0)` gives blocking calls with an upper bound. This is useful for UDP clients, which must implement their own retransmission, e.g. a DNS resolver retrying after a timeout.

12.3 The listen backlog: two kernel queues

Queue	Holds	Limit (Linux)	When full
SYN queue (incomplete)	Half-open connections in SYN_RCVD, waiting for the final ACK	tcp_max_syn_backlog	New SYNs are dropped, unless SYN cookies are enabled
Accept queue (complete)	ESTABLISHED connections not yet returned by accept()	min(backlog, somaxconn)	The final ACK is dropped and the client retransmits later, so a slow server shows up as connect delays

12.4 Useful socket options

`SO_RCVBUF / SO_SNDBUF`	Size of the receive and send buffers. RcvBuffer limits rwnd, and therefore throughput ≤ RcvBuffer / RTT
`TCP_NODELAY`	Disables Nagle's algorithm (interactive or low-latency applications, games)
`SO_REUSEADDR`	Allows binding a port that still has connections in TIME_WAIT (fast server restart)
`SO_KEEPALIVE`	Periodic probes on idle connections
`TCP_CONGESTION`	(Linux) Choose the congestion-control algorithm per socket: "cubic", "bbr", "reno"
`connect() on UDP`	Fixes a default peer, so send() and recv() can be used, datagrams from other addresses are filtered out, and ICMP errors (port unreachable) are reported to the application

12.5 Tools

Tool	What it shows about the transport layer
Wireshark / tcpdump	Every header field. Useful display filters: `tcp.flags.syn==1 && tcp.flags.ack==0` (connection attempts), `tcp.analysis.retransmission`, `tcp.analysis.fast_retransmission`, `tcp.analysis.duplicate_ack`, `tcp.analysis.zero_window`, `tcp.analysis.window_full`, `tcp.stream eq 3`, `udp.port==53`. Under Statistics: **Flow Graph** (a time-sequence ladder) and **TCP Stream Graphs** (Stevens and tcptrace plots of seq # vs time, Throughput, Round Trip Time, Window Scaling). **Trap**: Wireshark shows *relative* sequence numbers starting at 0 by default. Real ISNs are random 32-bit values.
ss / netstat	`ss -tan` lists TCP sockets and their states (LISTEN, SYN-SENT, ESTAB, TIME-WAIT, CLOSE-WAIT…). Many sockets stuck in **CLOSE_WAIT** means the application forgot to call close(). `ss -ti` shows the kernel's per-connection **cwnd, ssthresh, rtt/rttvar, rto, retrans**: the variables from this chapter, live.
iperf3	Measures TCP or UDP (`-u`) throughput, with `-P n` for parallel streams (the fairness experiment) and `-C bbr` to choose the congestion-control algorithm.
traceroute	The Unix version sends **UDP** probes to unlikely high ports (33434 and up), increasing the TTL. Routers return ICMP *time exceeded*. The destination has no socket on that port, so UDP demultiplexing fails and it returns ICMP **port unreachable**, which tells traceroute it is done. `-T` uses TCP SYNs (useful through firewalls), and `-I` uses ICMP echo.
nmap	`-sS` SYN ("half-open") scan: SYNACK = open, RST = closed, silence = filtered; the scanner never completes the handshake. `-sU` UDP scan: ICMP port unreachable = closed, silence = open or filtered.
netcat (nc)	Raw TCP/UDP client and server for experiments: `nc -l 12000`, `nc -u host 53`.

Wireshark’s limits

It sees one vantage point, so it cannot tell where a loss occurred. cwnd and ssthresh are **not in any header** and must be inferred from bytes in flight. Captures on the sender may show segments larger than the MSS because of TSO/GSO offload. With TLS over TCP, the TCP headers are visible but the payload is not. With QUIC, even the packet numbers and ACKs are encrypted unless the session keys are exported (SSLKEYLOGFILE).

13 · Transport-layer security issues

Your last paper asked about DNS security and why DoS needs botnets. These are the transport-layer equivalents.

13.1 SYN flooding

- **Attack**: send a torrent of SYNs, usually with **spoofed source IPs**, and never send the final ACK. Each SYN makes the server allocate a connection record (buffers and variables) in the SYN queue and send a SYNACK to a non-existent host. The SYN queue fills, and legitimate SYNs are dropped. The server's bandwidth is fine, but its **state** is exhausted.
- **Why spoofing works**: the attacker never needs the SYNACK. With UDP, or with the first TCP packet, nothing validates the source address.
- **SYN cookies** (Bernstein, 1996): the server keeps **no state** on a SYN. It sets its ISN to a cookie, roughly `hash(src IP, src port, dst IP, dst port, secret, time counter)` plus an encoded MSS, and sends the SYNACK. A genuine client's ACK carries ack = cookie + 1. The server recomputes the hash, and if it matches, it creates the connection only now. A spoofed SYN costs the server nothing more than one SYNACK. **Downside**: the options in the SYN (window scale, SACK) are lost unless encoded in the timestamp option.
- Other defences: larger SYN queues, shorter SYN_RCVD timeouts, SYN proxies and firewalls, rate limiting.

13.2 Why a DoS attacker needs many machines (botnets)

One attacker is limited by its **own access-link upload rate**, which is far below a data centre's capacity. Its traffic also comes from **one source**, which is easy to filter or block. A **botnet** of thousands of compromised hosts aggregates their bandwidth, spreads the traffic across many legitimate-looking source addresses and networks, and defeats simple per-source filtering. Transport-layer twists: SYN floods need no bandwidth advantage; **amplification** multiplies each bot's bandwidth; **Slowloris** opens many connections and sends headers very slowly, tying up server connection slots with tiny bandwidth.

13.3 UDP reflection and amplification

The attacker sends small UDP requests with the **source IP spoofed to be the victim's** to open servers, which send large responses to the victim. Because UDP has **no handshake**, the server cannot check who really asked. Amplification factors: DNS (especially ANY queries) about 28–54×, NTP *monlist* about 556×, memcached up to about 51,000×. Defences: **ingress filtering (BCP 38)** to stop spoofed packets at their source, response rate limiting, and closing open resolvers and servers. QUIC's **3× anti-amplification limit** exists for this reason.

13.4 Sequence-number attacks

- **RST injection**: an off-path attacker who can guess the 4-tuple and a sequence number **within the receive window** can kill a connection with a forged RST. This matters for long-lived BGP sessions, and it is used by some national firewalls to cut connections. The mitigation is RFC 5961: accept an RST only if its seq exactly matches the next expected byte; otherwise send a "challenge ACK".
- **Session hijacking / blind spoofing** (the 1994 Mitnick attack): predictable ISNs let an attacker complete a handshake while impersonating a trusted host. Hence **random ISNs** (RFC 6528).
- **TLS does not protect TCP headers**: TLS encrypts and authenticates the application data, but a forged RST or FIN still tears down the TCP connection. QUIC encrypts and authenticates almost the whole packet, so a middlebox cannot forge or read its control information.

13.5 Other issues

- **Port scanning** (nmap, §12.5) is reconnaissance: open ports reveal the running services and their versions.

- **Congestion-control cheating**: a misbehaving receiver can split its ACKs (“ACK division”) or send optimistic ACKs for data not yet received, inflating the sender’s cwnd and stealing bandwidth. There is no “Internet police”.
- **QUIC 0-RTT replay**: data sent in the first flight can be replayed by an attacker, so servers accept only idempotent requests in 0-RTT.

14 · Practice problem bank

These add to the 14 numericals in the Exam Prep guide, with no repeats. All arithmetic has been checked by computation. Try each problem before reading the answer.

A. Checksum, UDP, demultiplexing

P1 **8-bit 1’s-complement checksum**

Compute the checksum of the bytes 01010011, 01100110 and 01110100. Can a 1-bit error go undetected? What about a 2-bit error?

01010011 + 01100110 = 10111001. Adding 01110100 gives 1 00101101, and the wraparound gives **00101110** (the sum). The checksum is **11010001**. The receiver adds all four bytes and must get 11111111. **Every 1-bit error is detected**, because it changes the sum. A 2-bit error can go undetected if the two flips are in the same bit position in opposite directions.

P2 **16-bit checksum (K&R §3.3 example)**

Find the checksum of 0110011001100000, 0101010101010101 and 1000111100001100.

Word 1 + word 2 = 1011101110110101. Adding word 3 gives 1 0100101011000001, and the wraparound gives **0100101011000010**. The checksum is **1011010100111101**.

P3 **Demultiplexing**

Clients A and B each send UDP datagrams to server S on port 53. A uses source port 5000 and B also uses source port 5000. (a) How many sockets does S need? (b) How does S reply to the right client? (c) Repeat with TCP to port 80.

(a) **1** socket: UDP demultiplexes on the destination port only. (b) recvfrom() returns each datagram’s source (IP, port), and S sends the reply to that address. A and B are distinguished by **IP address**, even though their ports are equal. (c) TCP: **3** sockets, namely the listening socket plus one per connection, each identified by its 4-tuple (A, 5000, S, 80) and (B, 5000, S, 80).

B. rdt, stop-and-wait, pipelining

P4 **Conceptual rdt checks**

(a) Why doesn’t the rdt3.0 receiver need a timer? (b) Is rdt3.0 correct if the channel can *reorder* packets? (c) In rdt2.1, why must the receiver ACK a duplicate packet?

(a) Only the sender retransmits. The receiver just reacts to each arrival, and a lost ACK is recovered by the sender’s timer. (b) **No**. With 1-bit sequence numbers, a very old delayed packet with seq 0 could arrive while the receiver expects a *new* 0, and it would be accepted as new data. The sequence space would need to be larger than the number of packets that can be in the network at once. (c) A duplicate means the sender never got our ACK. Without a new ACK, it would retransmit forever (deadlock).

P5 **Stop-and-wait vs window**

A 10 Mbps link carries 1000-byte packets with a 20 ms one-way propagation delay, and ACK transmission time is negligible. Find the stop-and-wait utilisation and the window needed for 100% utilisation.

L/R = 8000 / 10^7 = 0.8 ms, and RTT = 40 ms. U = 0.8 / 40.8 = **1.96%**. The window needed is 40.8 / 0.8 = **51 packets**.

P6 **GBN window positions (K&R P22)**

GBN with N = 4. The receiver is expecting packet k, and there is no reordering. (a) What sender windows are possible? (b) Which ACK values can be in flight to the sender?

(a) If all ACKs up to k − 1 have reached the sender, its window is [k, k+3]. If none of the last N ACKs has arrived, it is [k−4, k−1]. So the window starts anywhere in **[k−4, k]**. (b) The receiver has ACKed up to k − 1, and the sender must already have received the ACK for k − 5 (otherwise it could not have sent k − 1). So the in-flight ACKs lie in **[k−5, k−1]**.

P7 **Sequence bits**

What is the minimum number of seq # bits for a window of N = 5 with GBN, and with SR?

GBN: $2^k - 1 \geq 5$, so **k = 3**. SR: $2^{k-1} \geq 5$, so **k = 4**.

P8 **GBN vs SR with a lost packet *and* a lost ACK**

N = 5, and packets 0–9 are to be sent. Packet 3 is lost once, and the ACK for packet 6 is lost once. Count the transmissions under GBN and under SR.

GBN: send 0–4 (3 is lost). ACKs 0–2 move the window to 3–7, so send 5, 6 and 7, all of which the receiver discards. The timeout for 3 fires, so resend **3, 4, 5, 6, 7**. ACK6 is lost, but **the cumulative ACK7 covers it**, and packets 8 and 9 are then sent. Total = 10 + 5 = **15**.
SR: only 3 is resent. ACK6 is lost, and ACKs are individual, so **packet 6's timer expires and 6 is resent**. The receiver re-ACKs it, since it is in [rcvbase − N, rcvbase − 1]. Total = 10 + 2 = **12**.
Lesson: SR saves bandwidth on data loss, but a cumulative ACK is more robust to *ACK* loss.

C. TCP sequence numbers, RTT, timers

P9 Full connection trace

The client's ISN is 1000 and the server's ISN is 5000. The client sends a 300-byte request piggybacked on the third handshake segment, the server replies with 1200 bytes in one segment, the client ACKs, and then each side closes (client first). List seq and ack for every segment.

#	Segment	seq	ack	Data bytes
1	C→S SYN	1000	—	0 (SYN consumes 1)
2	S→C SYNACK	5000	1001	0 (SYN consumes 1)
3	C→S ACK + request	1001	5001	300 (1001–1300)
4	S→C response	5001	1301	1200 (5001–6200)
5	C→S ACK	1301	6201	0
6	C→S FIN	1301	6201	0 (FIN consumes 1)
7	S→C ACK	6201	1302	0
8	S→C FIN	6201	1302	0 (FIN consumes 1)
9	C→S ACK	1302	6202	0 → client enters TIME_WAIT

P10 Segment sizes from sequence numbers

A sends three segments with seq 90, 110 and 150, and the third carries 30 bytes. (a) How many bytes do the first two carry? (b) The first is lost and the other two arrive. What ACKs does B send? (c) After A retransmits, what is the ACK?

(a) 110 − 90 = **20 bytes**, and 150 − 110 = **40 bytes**. (b) **ACK 90** twice (duplicates); B buffers the out-of-order bytes. (c) Once the gap is filled, the cumulative ACK is 150 + 30 = **180**.

P11 EWMA over several samples

EstimatedRTT = 100 ms and DevRTT = 5 ms initially, with α = 0.125 and β = 0.25. The SampleRTTs are 106, 120, 140, 90 and 115 ms. Give the timeout after each sample (update EstimatedRTT first, then DevRTT using the new estimate).

Sample	106	120	140	90	115
EstimatedRTT	100.75	103.16	107.76	105.54	106.72
DevRTT	5.06	8.01	14.07	14.43	12.89
Timeout	**121.0**	**135.2**	**164.0**	**163.3**	**158.3**

Note that the 90 ms sample *lowers* EstimatedRTT but *raises* DevRTT, so the timeout barely falls. Variability is penalised.

P12 Timer backoff

TimeoutInterval = 0.75 s, and the same segment times out three times in a row. What are the successive timeout values?

It doubles each time: **1.5 s, 3 s, 6 s**. Once an ACK arrives for new, non-retransmitted data, the timeout is recomputed from EstimatedRTT + 4·DevRTT.

P13 **Sequence-number wrap**

How long does the 32-bit sequence space take to wrap at 1 Gbps, and at 100 Gbps? Why does it matter?

$2^{32} \times 8 / 10^9 \approx$ **34 s**; at 100 Gbps ≈ **0.34 s**. That is less than the MSL (about 2 min), so a delayed old segment could fall inside the current window and be accepted. The fix is **PAWS**, which uses the timestamps option to reject old segments.

D. Flow control and windows

P14 **rwnd**

RcvBuffer = 65,536 B. The receiver has received up to byte 140,000, and the application has read up to byte 100,000. What is rwnd? If the sender's LastByteAcked = 130,000, what is the largest LastByteSent allowed?

rwnd = 65,536 − (140,000 − 100,000) = **25,536 B**. LastByteSent ≤ LastByteAcked + rwnd = **155,536**. (The ACK for bytes up to 140,000 is still in flight to the sender.)

P15 **Window scale**

What window, and what window-scale shift, are needed to fill a 10 Gbps path with a 60 ms RTT?

BDP = $10^{10} \times 0.06 / 8$ = **75 MB**. $75 \times 10^6 / 65{,}535 \approx 1144$, so the scale factor must be at least 1144. The next power of two is $2^{11} = 2048$, so the **shift is 11**.

E. Congestion control

P16 **Tahoe vs Reno after a triple duplicate ACK**

cwnd = 24 MSS when 3 duplicate ACKs arrive. Give cwnd for the next 5 RTTs under Tahoe, Reno (simplified) and Reno with fast recovery. Assume recovery completes within one RTT.

ssthresh = 12 in all cases. **Tahoe**: 1, 2, 4, 8, 12 (slow start capped at ssthresh), then 13. **Reno (simplified)**: 12, 13, 14, 15, 16. **Reno with FR**: cwnd = 15 during fast recovery (inflating by 1 per extra dup ACK), then it deflates to 12 on the new ACK, followed by 13, 14, 15, 16.

P17 **Slow-start arithmetic**

Starting from cwnd = 1 with no loss, how many RTTs until cwnd = 64, and how many segments have been sent by then?

cwnd = 2^n after n RTTs, so **6 RTTs**. Segments sent = 1 + 2 + … + 32 = **63**.

P18 **Sawtooth on one link (K&R P45-style)**

A single Reno flow uses a 10 Mbps link with RTT = 150 ms and MSS = 1500 B. Loss occurs exactly when the window reaches the link's capacity. (a) What is W at loss (in segments)? (b) What is the average throughput? (c) How long does it take to grow from W/2 back to W?

(a) $W = R \cdot RTT / MSS = 10^7 \times 0.15 / 12{,}000$ = **125 segments**. (b) 0.75 × 10 Mbps = **7.5 Mbps**. (c) 62.5 RTTs × 0.15 s ≈ **9.4 s**. The router needs a buffer of about one BDP to keep the link busy through each halving, which is the buffer = RTT × C rule.

P19 **Mathis formula**

MSS = 1460 B, RTT = 80 ms, loss rate L = 10^{-4}. What is the throughput? What loss rate is needed for 100 Mbps?

1.22 × 11,680 bits / (0.08 × 0.01) ≈ **17.8 Mbps**. For 100 Mbps: L = (1.22 × 11,680 / (0.08 × 10^8))² ≈ **3.2 × 10^{-6}**. Halving the RTT would double throughput at the same loss rate.

P20 **RTT unfairness**

Two Reno flows share a 12 Mbps bottleneck and see the same loss rate. Their RTTs are 50 ms and 150 ms. How is the bottleneck shared?

Throughput ∝ 1/RTT, so the ratio is 3 : 1: **9 Mbps and 3 Mbps**.

P21 **Parallel connections**

A 10 Mbps bottleneck carries 4 TCP connections. A download manager opens 6 more. What does it get?

6/10 × 10 = **6 Mbps**, while each of the 4 others drops to 1 Mbps.

P22 **Download time with slow start**

RTT = 50 ms, MSS = 1 KB, a 20 KB object, transmission time negligible, no loss. Find the time from connect() to the last byte with IW = 1, and with IW = 10.

IW = 1: windows of 1, 2, 4, 8 and then 5 (20 segments) take 5 RTTs, plus 1 RTT for the handshake, giving 6 RTT = **300 ms**. IW = 10: windows of 10 and 10 take 2 RTTs, plus the handshake, giving 3 RTT = **150 ms**.

P23 **CUBIC numbers**

W_{max} = 100, C = 0.4, β = 0.3. Find cwnd right after the loss, K, and W(2 s) and W(6 s).

cwnd = 0.7 × 100 = **70**. K = $\sqrt[3]{(100 \times 0.3 / 0.4)} = \sqrt[3]{75} \approx$ **4.22 s**. W(2) = 0.4(2 − 4.22)³ + 100 ≈ **95.6**. W(6) = 0.4(1.78)³ + 100 ≈ **102.3**. It grew quickly back to near W_{max} and is now probing slowly beyond it.

P24 **Congestion scenario 2 arithmetic**

In the “known loss” case, offered load λ'_{in} = R/2 and goodput = R/3. What fraction of the transmitted bytes are retransmissions?

Retransmitted = R/2 − R/3 = R/6, so the fraction is (R/6) / (R/2) = **1/3** of everything sent.

P25 **Queueing delay link (Q10 from your last paper)**

A router has a 1 MB buffer on a 100 Mbps link. What is the maximum queueing delay, and why does it matter for TCP?

8 × 10^6 bits / 10^8 bps = **80 ms**. Loss-based TCP fills the buffer before it sees a loss, so every packet on the path waits up to 80 ms: bufferbloat. Delay-based CC (BBR, Vegas) and AQM (CoDel, RED with ECN) keep the queue short.

F. QUIC, latency, Nagle

P26 **Page-load setup latency**

DNS lookup = 20 ms (UDP, cached at the local resolver) and RTT to the server = 60 ms. What is the time to the first response byte with TCP + TLS 1.3, QUIC (1-RTT) and QUIC 0-RTT?

TCP + TLS 1.3: 20 + 60 (TCP) + 60 (TLS) + 60 (request) = **200 ms**. QUIC: 20 + 60 (combined handshake) + 60 (request) = **140 ms**. QUIC 0-RTT: 20 + 60 = **80 ms**.

P27 **Head-of-line blocking**

Three HTTP/2 streams (A, B, C) each have one packet in flight. A's packet is lost and is recovered one RTT (100 ms) later. When are B and C delivered to the application over TCP, and over QUIC?

TCP: B's and C's bytes sit *after* the hole in the single byte stream, so both wait about **+100 ms**. QUIC: B and C are delivered **immediately**, and only A waits.

P28 **Nagle's algorithm**

A telnet user types 10 characters, one every 100 ms (t = 0, 100, …, 900 ms). RTT = 250 ms. How many segments are sent with Nagle, and without it?

With Nagle: t = 0 carries 1 char. The ACK returns at 250 and carries the chars typed at 100 and 200. At 500: chars at 300 and 400. At 750: chars at 500, 600 and 700. At 1000: chars at 800 and 900. That is **5 segments** (1 + 2 + 2 + 3 + 2 chars). Without Nagle: **10 segments**, each with about 40 bytes of headers for 1 byte of data.

P29 **Throughput ceiling from rwnd**

A receiver fixes RcvBuffer = 256 KB (262,144 B), and RTT = 40 ms. What is the maximum throughput on a 1 Gbps path?

262,144 × 8 / 0.04 ≈ **52 Mbps**. Flow control caps the transfer at about 5% of the link. A larger buffer, or auto-tuning, is needed.

P30 **Header efficiency**

What is the goodput fraction for 1460-byte TCP payloads over Ethernet (a 14 + 4 byte frame header and trailer, plus 20-byte preamble and inter-frame gap)? And for 1472-byte UDP payloads?

TCP: 1460 / (1460 + 20 + 20 + 18 + 20) = 1460 / 1538 = **94.9%**. UDP: 1472 / (1472 + 8 + 20 + 38) = 1472 / 1538 = **95.7%**. The difference is small for bulk transfers; UDP's header advantage matters only for small packets.

15 · Rapid-fire recall: 60 one-liners

Cover the right-hand side and answer aloud. If you can do all 60, the 2-mark questions are safe.

#	Question	Answer
1	Transport vs network layer: communication between?	Processes vs hosts
2	Where does the transport layer run?	End systems only
3	Services neither TCP nor UDP provides?	Delay guarantees, bandwidth guarantees
4	UDP demultiplexing key?	Destination port (and destination IP)
5	TCP demultiplexing key?	4-tuple: source IP, source port, destination IP, destination port
6	Size of a port number? Well-known range?	16 bits; 0–1023
7	UDP header size and fields?	8 B: source port, destination port, length, checksum
8	What does the UDP length field count?	Header + data, in bytes
9	Checksum arithmetic?	16-bit 1's-complement sum with wraparound, then inverted
10	What does the checksum cover besides the segment?	The pseudo-header (IP addresses, protocol, length)
11	Is the UDP checksum optional?	In IPv4 yes (0 = none); in IPv6 no
12	Four reasons to use UDP?	No setup RTT, no connection state, small header, no congestion throttling (application controls timing)
13	Which rdt version adds sequence numbers?	rdt2.1
14	Which rdt version is NAK-free?	rdt2.2
15	Which rdt version adds a timer? Which impairment does it handle?	rdt3.0; packet loss
16	rdt2.0's fatal flaw?	Corrupted ACK/NAK: the sender can't tell what happened, and a blind retransmission creates duplicates
17	Another name for rdt3.0?	Alternating-bit protocol
18	How many sender states in rdt2.1 and rdt3.0?	4
19	What the rdt3.0 sender does on a duplicate ACK?	Nothing (Λ). It waits for the timeout
20	Stop-and-wait utilisation formula?	(L/R) / (RTT + L/R)
21	Two requirements of pipelining?	A larger seq # range; buffering at the sender and/or receiver
22	GBN ACK type and number of timers?	Cumulative; one (for the oldest unACKed packet)
23	SR ACK type and number of timers?	Individual; one per unACKed packet
24	What the GBN receiver does with out-of-order packets?	Discards them (or buffers, by implementation choice) and re-ACKs the highest in-order packet
25	SR receiver: packet in [rcvbase−N, rcvbase−1]?	ACK it again
26	Maximum window, k-bit seq #: GBN and SR?	$2^k - 1$; 2^{k-1}
27	What TCP seq # counts?	Bytes: the first data byte of the segment
28	What the TCP ACK number means?	The next byte expected (cumulative)
29	Typical MSS? How it is derived?	1460 B = 1500 MTU − 20 IP − 20 TCP
30	TCP header size range?	20–60 B (header length field × 4)
31	TCP flag bits (8)?	CWR, ECE, URG, ACK, PSH, RST, SYN, FIN
32	Size of the receive-window field?	16 bits (max 65,535 B without scaling)

33	How TCP handles out-of-order segments?	Not specified by the standard; implementations buffer them
34	α and β in RTT estimation?	0.125 and 0.25
35	TimeoutInterval formula?	EstimatedRTT + 4·DevRTT
36	What happens to the timeout after a timeout?	It doubles (exponential backoff)
37	Why retransmitted segments are excluded from SampleRTT?	The ACK is ambiguous (Karn's algorithm)
38	Maximum delayed-ACK wait?	500 ms
39	Fast retransmit trigger?	3 duplicate ACKs (4 identical ACKs in total)
40	Is TCP GBN or SR?	A hybrid: cumulative ACK and one timer (GBN), but it buffers and resends one segment (SR); SACK brings it close to SR
41	rwnd formula?	RcvBuffer − (LastByteRcvd − LastByteRead)
42	What the sender does when rwnd = 0?	Sends 1-byte probes (persist timer)
43	Default RcvBuffer on the slides?	4096 bytes
44	Two things agreed in the handshake?	Willingness to connect; parameters (initial seq #s, buffer sizes)
45	Two failure modes of a 2-way handshake?	Half-open connection; duplicate old data accepted
46	SYNACK fields?	SYN = 1, ACK = 1, seq = y, ack = x + 1
47	Can the 3rd handshake segment carry data?	Yes
48	Do SYN and FIN consume sequence numbers?	Yes, one each
49	Client states during close?	FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED
50	Server states during close?	CLOSE_WAIT → LAST_ACK → CLOSED
51	Duration of TIME_WAIT?	2 × MSL (30 s – 2 min)
52	Three costs of congestion?	Large delays; wasted retransmissions and duplicates; upstream capacity wasted on packets dropped downstream
53	Two approaches to congestion control?	End-end (TCP) and network-assisted (ECN, ATM, DECbit)
54	AIMD increase and decrease?	+1 MSS per RTT; halve on loss
55	Slow start growth and implementation?	Doubles per RTT; cwnd += 1 MSS per ACK
56	ssthresh on loss?	cwnd/2
57	Tahoe vs Reno on 3 duplicate ACKs?	Tahoe: cwnd = 1. Reno: cwnd = ssthresh (+3) and fast recovery
58	Average throughput formula? Mathis formula?	0.75·W/RTT; 1.22·MSS/(RTT√L)
59	CUBIC's growth function and where it is the default?	$C(t - K)^3 + W_{max}$; Linux
60	QUIC: which layer, over what, handshake cost, key benefit?	Application layer, over UDP; 1 RTT (0-RTT on resume); streams with no HOL blocking

16 · Glossary

ACK clocking	TCP releases new segments as ACKs return, so the sending rate self-adjusts to the bottleneck rate.
AIMD	Additive increase, multiplicative decrease: +1 MSS per RTT, halve on loss.
AQM	Active queue management (RED, CoDel): routers drop or mark packets early to keep queues short.
ARQ	Automatic Repeat reQuest: error control by detection, feedback (ACK/NAK) and retransmission.
BBR	Google's model-based congestion control, which estimates bottleneck bandwidth and minimum RTT and paces at that rate.
BDP	Bandwidth-delay product R × RTT: the amount of data needed in flight to fill the pipe.
Bottleneck link	The link on a path with the lowest available capacity; where the queue builds.
Bufferbloat	Excessive delay caused by over-large router buffers that loss-based TCP keeps full.
Congestion collapse	A state where offered load rises but goodput falls toward zero, because capacity is spent on retransmissions and packets dropped downstream.
cwnd	Congestion window: the sender's private limit on unACKed bytes, set by congestion control.
CUBIC	The default Linux TCP. Its window follows a cubic function of the time since the last loss, centred on W_{max}.
Cumulative ACK	An ACK that acknowledges all data up to a point (GBN, TCP).
Delayed ACK	The receiver waits up to 500 ms before ACKing, so it can combine ACKs or piggyback them on data.
Demultiplexing	Delivering received segments to the correct socket using header fields.
DevRTT	EWMA of |SampleRTT − EstimatedRTT|: the variability of the RTT.
Duplicate ACK	A re-ACK of the same next-expected byte, which signals a gap at the receiver.
ECN / ECE / CWR	Explicit Congestion Notification. A router sets CE in the IP header, the receiver echoes it with ECE in the TCP header, and the sender confirms its reduction with CWR.
Ephemeral port	A temporary client-side port (49152–65535) assigned by the OS.
EstimatedRTT	The smoothed RTT, an EWMA of SampleRTT with α = 0.125.
EWMA	Exponentially weighted moving average: the weight of old samples decays geometrically.
Fast recovery	The Reno state after 3 duplicate ACKs: cwnd = ssthresh + 3, inflated by 1 per extra duplicate ACK, and deflated on a new ACK.
Fast retransmit	Resending the missing segment on 3 duplicate ACKs, before the timer expires.
FIN	The flag that closes one direction of a TCP connection. It consumes one sequence number.
Flow control	Matching the sender's rate to the receiver's buffer, via rwnd.
FSM	Finite state machine: states plus event/action transitions, used to specify protocols.
Go-Back-N	A pipelined protocol with cumulative ACKs and one timer; on a timeout it resends every unACKed packet.
Goodput	The useful data delivered to the application per second (excluding retransmissions and headers).
Half-open connection	A connection that one side believes is open while the other side does not (or no longer exists).
HOL blocking	Head-of-line blocking: data behind a missing piece cannot be delivered even if it belongs to an unrelated stream.
ISN	Initial sequence number, chosen randomly by each side during the handshake.

Karn's algorithm	Take no RTT samples from retransmitted segments, and keep the backed-off timer until a fresh sample arrives.
Listen backlog	A bound on the queue of connections that have completed the handshake but not yet been accepted.
MSL	Maximum segment lifetime: the assumed maximum time a segment can survive in the network (about 2 min).
MSS	Maximum segment size: the largest application payload per TCP segment.
MTU	Maximum transmission unit: the largest link-layer payload (1500 B for Ethernet).
Multiplexing	Gathering data from many sockets and adding transport headers so it can be demultiplexed later.
Nagle's algorithm	Coalesce small writes while data is unACKed, to avoid tiny segments.
Offered load (λ'_{in})	Original data plus retransmissions sent into the network.
PAWS	Protection Against Wrapped Sequence numbers, using the TCP timestamps option.
Piggybacking	Carrying an ACK in a data segment travelling in the reverse direction.
Pipelining	Having several unACKed packets in flight at once.
Pseudo-header	IP addresses, protocol and length included in the TCP/UDP checksum computation.
QUIC	A UDP-based, encrypted, multi-stream transport with 1-RTT setup, used by HTTP/3.
rdt	The textbook's family of reliable data transfer protocols (1.0, 2.0, 2.1, 2.2, 3.0).
RST	The TCP flag that aborts a connection, or rejects a SYN to a closed port.
rwnd	Receive window: the free buffer space advertised by the receiver.
SACK	Selective acknowledgement option (RFC 2018): reports the non-contiguous blocks received.
SampleRTT	The measured time from sending a (non-retransmitted) segment to receiving its ACK.
Selective Repeat	A pipelined protocol with individual ACKs, per-packet timers and receiver buffering; it resends only the lost packets.
Slow start	The exponential cwnd growth phase (doubling per RTT) from 1 MSS up to ssthresh.
Socket	The interface between an application process and the transport layer, identified by its port (UDP) or 4-tuple (TCP connection).
ssthresh	Slow-start threshold: the cwnd value at which TCP switches from exponential to linear growth; set to cwnd/2 on loss.
Stop-and-wait	Sending one packet and waiting for its ACK before sending the next (rdt2.x, rdt3.0).
SYN / SYN cookie	SYN: the connection-request flag. SYN cookie: a stateless server ISN that encodes the connection parameters, a defence against SYN floods.
TIME_WAIT	The 2·MSL wait by the active closer, to re-ACK a lost final FIN and let old duplicates expire.
Triple duplicate ACK	Three duplicate ACKs, taken as a loss signal (fast retransmit).
Window scaling	A TCP option multiplying rwnd by 2^{shift} (shift ≤ 14), allowing windows above 64 KB.

Sources: Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Chapter 3 slides v8.2 (including speaker notes) and textbook; RFC 768 (UDP), 793/9293 (TCP), 1122, 2018 (SACK), 3168 (ECN), 5681 (TCP congestion control), 5961, 6298 (RTO), 6528 (ISN), 6582 (NewReno), 6928 (IW10), 7323 (window scale, timestamps), 7413 (TFO), 9000–9002 (QUIC), 9114 (HTTP/3), 9438 (CUBIC); Chiu & Jain (1989); Mathis et al. (1997); Cardwell et al., "BBR" (2016).